



REQUI TRADING · ENGINEERING HANDOFF

Autonomous Strategy Engine Add-On — Implementation Guide

Modules 0–6: IBKR live data feed · trigger library · setup state machine ·
backtester · what-if simulator · scanner · confirmation-gated execution

Audience: DevOps / backend engineer merging this add-on into the
previous Requi Trading codebase

Contents: architecture, file placement map, integration steps,
operations notes, full source code (Appendices A–P)

Compatibility: React 19 + Vite frontend · tRPC 11 + Hono API ·
Drizzle ORM · same layout as the existing repo



Contents

1. What this add-on is
2. Architecture — how it works
3. File placement map
4. Integration steps
5. Module behavior — what serves what
6. Operations: environment, IBKR gateway, failure modes
7. Safety invariants (non-negotiable)
 - Appendix A — `api/marketdata/indicators.ts`
 - Appendix B — `api/marketdata/ibkr-data.ts`
 - Appendix C — `api/marketdata/service.ts`
 - Appendix D — `api/marketdata-router.ts`
 - Appendix E — `api/engine/triggers.ts`
 - Appendix F — `api/engine/config.ts`
 - Appendix G — `api/engine/state-machine.ts`
 - Appendix H — `api/engine/risk.ts`
 - Appendix I — `api/engine/backtest.ts`
 - Appendix J — `api/engine/simulator.ts`
 - Appendix K — `api/engine/scanner.ts`
 - Appendix L — `api/engine/monitor.ts`
 - Appendix M — `api/engine-router.ts`
 - Appendix N — `src/components/DataFeedPanel.tsx`
 - Appendix O — `src/components/EnginePanel.tsx`
 - Appendix P — Edits to existing files (`router.ts`, `Autonomous.tsx`)

1. What this add-on is

This package adds the **Autonomous Strategy Engine** to the existing Requi Trading platform. It is an add-on, not a rebuild: it reuses the platform's governance compiler/runtime (signed package), the order-ticket confirmation gate, the broker adapters (Paper / Robinhood MCP / IBKR), the alerts system, and the Autonomous page. Nothing in those systems is rewritten — the engine plugs into them.

Seven modules, built in dependency order:

Module	Name	What it does	Files
0	Live data feed ("Eyes")	Pulls 1-minute OHLCV bars from the IBKR Client Portal gateway, refreshes every 60s during market hours (09:30–16:00 ET), computes session VWAP, RSI(14), prior-day high/low/close, session high/low.	Appendices A–D
1	Trigger library	9 reusable, pure trigger functions (vwap_reclaim, pullback_hold, double_bottom, breakout, volume_climax, rsi_divergence, relative_strength, prior_day_high_reclaim, gap_fill). Each returns armed + the evidence that armed it.	Appendix E
2	Setup state machine ("Brain")	IDLE → WATCHING → CONFIRMED → IN_POSITION → OUT / KILLED per symbol. Timer counts consecutive 1-min closes above the zone floor; any close below resets it to 0 — full reset, never a pause. Ranked entry-variant ladder with OCO invalidation.	Appendix G
3	Backtester	Replays 1-min bars day-by-day through the <i>same</i> state machine. No lookahead: entry fills are evaluated on the bar after confirmation; stops fill before targets within a bar. Full report: win rate, profit factor, expectancy (R), max drawdown, per-trade log.	Appendix I
4	What-if simulator	Runs the backtester per entry variant, ranks variants by expectancy, drops negative-expectancy variants, and outputs the re-ranked fallback ladder used live.	Appendix J
5	Scanner hook	Scans the tracked watchlist for candidates (gap %, relative volume, position vs VWAP) and selects the best-fit strategy config per candidate.	Appendix K
6	Execution wiring ("Hands")	Risk sizing from (entry – stop) at $\leq 0.5\%$ account risk and $\leq 25\%$ position cap; CONFIRMED setups become order tickets through the existing confirmation gate. Phase 1: no auto-execution — every proposal waits for CONFIRM ORDER [TICKET_ID].	Appendices H, L

Config-driven design: strategies are data (Appendix F), not code. Adding a strategy means adding a config object — entry variants, stop rule, targets, hold timer, sizing, schedule. The engine runs any config on any symbol. No ticker or price is hardcoded anywhere.

2. Architecture — how it works

2.1 The autonomous loop



The data feed (module 0) refreshes tracked symbols every 60 seconds during regular market hours. On demand (UI "Evaluate now", or after each refresh tick), each active monitor feeds the freshest session bars into its state machine. When the machine reaches CONFIRMED, the monitor sizes the position through the risk engine and stages an order ticket via the platform's existing `proposeTicket()`. The ticket lands in the same confirmation flow used by the Intelligence chat: the user types `CONFIRM ORDER [TICKET_ID]` and only then does the broker adapter submit. The engine never touches a broker directly.

2.2 One engine, three modes

The single most important design rule: **live, backtest, and simulation all call the same `SetupMachine.onBar(ctx)` function.** Only the bar source differs — live feed (module 0), historical replay (module 3), or per-variant replay (module 4). The machine only ever receives bars up to "now", so lookahead bias is impossible by construction. This is what makes backtest results meaningful for live trading.

2.3 Governance clamping

Every strategy config is loaded through `loadConfig()` (Appendix F), which reads the platform's signed governance package and clamps any value that exceeds constitutional ceilings — e.g. a config requesting 2% risk is clamped to the package's 0.50% ceiling, and the clamp is recorded and surfaced. Configs can tighten the constitution, never loosen it. If no signed package is active, config loading throws `GOVERNANCE_UNAVAILABLE` and the engine refuses to operate — the constitution is load-bearing.

2.4 Data honesty

If the IBKR gateway is unreachable or unauthenticated, feeds carry the error verbatim and the UI displays it. The system never fabricates bars, never silently falls back to simulated data, and reports the broker-reported feed delay so timer logic can refuse to arm on stale data.

3. File placement map

All paths are relative to the repo root. Fifteen files are **new**; two existing files receive small edits (Appendix P).

#	Path	Layer	Purpose	Depends on
A	api/marketdata/indicators.ts	API · module 0	Indicator engine (pure functions)	nothing (std lib only)
B	api/marketdata/ibkr-data.ts	API · module 0	IBKR market-data endpoints	A
C	api/marketdata/service.ts	API · module 0	Watchlist + 60s refresh loop + cache	A, B
D	api/marketdata-router.ts	API router	tRPC surface for module 0	C + existing api/middleware.ts
E	api/engine/triggers.ts	API · module 1	Trigger library	A
F	api/engine/config.ts	API · config	Strategy configs + governance clamping	E + existing api/governance/runtime.ts
G	api/engine/state-machine.ts	API · module 2	Setup state machine	A, E, F
H	api/engine/risk.ts	API · module 6	Risk sizing / halt / chase-cancel	nothing
I	api/engine/backtest.ts	API · module 3	Backtester	A, F, G, H
J	api/engine/simulator.ts	API · module 4	Variant expectancy ranking	F, I
K	api/engine/scanner.ts	API · module 5	Candidate scanner	A, C, F
L	api/engine/monitor.ts	API · module 6	Live monitor → ticket gate	C, F, G, H + existing api/queries/tickets.ts
M	api/engine-router.ts	API router	tRPC surface for modules 1–6	E–L
N	src/components/DataFeedPanel.tsx	React UI	Live data feed panel	D via @/providers/trpc
O	src/components/EnginePanel.tsx	React UI	Engine panel (backtest/simulate/scan/monitor)	M via @/providers/trpc
P	api/router.ts · src/pages/app/Autonomous.tsx	edits	Router registration + panel mount	—

Prerequisites that must already exist in the previous version (they do in the merged handoff)

codebase): `api/middleware.ts` exporting `authedQuery/createRouter`; `api/governance/runtime.ts` exporting `loadActivePackage`; `api/queries/tickets.ts` exporting `proposeTicket` and the `PublicTicket` type; the tRPC client at `@/providers/trpc`; and the UI primitives used by the panels (`lucide-react`, `@/lib/utils` `cn`).

4. Integration steps

1. **Copy the new files.** Place Appendices A–C into `api/marketdata/`, Appendices E, G, H, I, J, K, L into `api/engine/`, Appendices D and M into `api/`, and Appendices N–O into `src/components/`. Directory contents must match the placement map in Section 3 exactly.
2. **Register the routers.** Apply the Appendix P edit to `api/router.ts`: import `marketDataRouter` and `engineRouter` and add `marketData` and `engine` keys to `appRouter`. This is what exposes `trpc.marketData.*` and `trpc.engine.*` to the frontend.
3. **Mount the panels.** Apply the Appendix P edit to `src/pages/app/Autonomous.tsx`: import both components and render them directly after the `RunConsole` block.
4. **Set environment variables** (see Section 6): `IBKR_GATEWAY_URL`, `IBKR_ACCOUNT`, and the existing `GOVERNANCE_VAULT_KEY` must be present. The engine refuses to run without an active signed governance package — `boot` already ensures one via `ensureGovernanceReady()`.
5. **Rebuild and restart.** `npm run build` (Vite + esbuild bundle) then restart the production server (`NODE_ENV=production node dist/boot.js`). Type-check first with `npx tsc --noEmit` — this codebase sets `erasableSyntaxOnly` (no constructor parameter properties) and `noUnusedLocals`; the add-on complies with both.
6. **Verify.** Unauthenticated `GET /api/trpc/engine.triggers` and `/api/trpc/marketData.status` must return `401 UNAUTHORIZED` (proves the routes exist and are auth-gated). Then log in, open the Autonomous page, and confirm both panels render: the data panel should show gateway status (unreachable is expected until step 7 of Section 6 is done), and the engine panel should list the three built-in configs.

Deployment note: no database migration is required for this add-on. Module 0 keeps bars in memory by design; tickets, alerts, and governance use existing tables created by the platform's boot-time `ensureSchema()`.

5. Module behavior — what serves what

5.1 Module 0 — data feed (Appendices A–D)

`marketDataService.track(symbol)` adds a symbol, starts the shared 60-second loop (ticks only Mon–Fri 09:30–16:00 ET), and immediately pulls 2 days of 1-minute bars from the IBKR gateway. `computeIndicators()` then derives the per-session set: VWAP (cumulative over the current session only — resets daily by construction), Wilder RSI(14) on 1-min closes, session high/low, and prior-day high/low/close. Bars are filtered to regular hours and grouped by ET calendar day via `groupRthSessions()` — every downstream module consumes these sessions.

5.2 Module 1 — triggers (Appendix E)

Each trigger is `(ctx, params) → { armed, evidence }` where `ctx` is the session bars + prior-day levels (+ optional benchmark bars for relative strength). Hold-based triggers (`vwap_reclaim`, `pullback_hold`) count *trailing consecutive closes above the floor* — a close below makes the count zero, which is the full-reset rule implemented as data, not state.

5.3 Module 2 — state machine (Appendix G)

`SetupMachine.onBar(ctx)` drives one symbol through IDLE → WATCHING (first close above floor) → timer accumulation (full reset on any close below) → CONFIRMED (timer $\geq$ threshold *and* variant trigger armed *and* signal stack $\geq$ minimum) → returns a `Proposal` (entry, stop, targets, evidence) exactly once. Chase-limit breaches invalidate the current variant and advance the ladder; all variants invalidated → NO_TRADE. After a fill (`notifyFill`, called by the broker path live or the replay in backtest), the same machine manages stops, targets, stop-to-breakeven ratcheting (tightening only), and the 15:55 time-flatten.

5.4 Modules 3–5 — backtest / simulate / scan (Appendices I–K)

The backtester groups bars into sessions and replays them through a fresh machine per day, applying risk-sized positions; entry stops are fill-checked on the bar *after* confirmation (no lookahead) and chase-canceled if the open gaps beyond the limit. The simulator runs the backtester once per variant and re-ranks the ladder by expectancy. The scanner turns tracked feeds into candidates (gap % ≥ 2 or RVOL ≥ 1.5) with a rule-based config assignment: down-gap below VWAP → bottom-fish config; up-gap/strength → continuation; otherwise general momentum.

5.5 Module 6 — monitor (Appendix L)

`startMonitor(userId, symbol, strategyId)` loads the governance-clamped config and creates a machine; `evaluateMonitor()` refreshes the feed, feeds the latest bar, and on CONFIRMED sizes the position (`sizePosition: qty = equity × 0.5% ÷ (entry – stop)`, capped at 25% of equity) and stages a ticket with `origin: "AUTONOMOUS"`. One proposal per symbol per day — the machine's CONFIRMED state plus the monitor's per-day latch provide the OCO behavior. The ticket then follows the platform's existing flow: alert with the exact confirm string, 5-minute TTL, idempotency key, broker submission only after confirmation.

6. Operations: environment, IBKR gateway, failure modes

6.1 Environment variables

Variable	Required	Purpose
IBKR_GATEWAY_URL	for live data	Client Portal Gateway base URL (default <code>https://localhost:5000/v1/api</code>). One gateway serves both trading and market data.
IBKR_ACCOUNT	for trading	IBKR account id used by the broker adapter.
GOVERNANCE_VAULT_KEY	yes (existing)	Vault key for the governance package. Config loading throws without an active signed package — engine refuses to run.
OPENAI_API_KEY	optional (existing)	Intelligence chat only. The engine itself is fully deterministic and needs no LLM.

6.2 IBKR gateway

The gateway is authenticated by the operator out-of-band (IBKR login + 2FA); the add-on never logs in programmatically. Health is checked via `GET /iserver/auth/status` every 30s and surfaced in the data panel. With a funded account and market-data subscriptions, 1-minute history is real-time; if IBKR reports delayed data, the feed's `delaySeconds` field shows it.

6.3 Expected behavior and failure modes

- **No gateway:** data panel shows the connection error per symbol; backtest/simulate return "Not enough bars"; monitors idle in IDLE. This is intended honest degradation.
- **Market closed:** the refresh loop does not tick; manual refresh still works and shows the last session.
- **Governance unavailable:** config load throws `GOVERNANCE_UNAVAILABLE`; nothing evaluates. Fix the package first — this is by design.
- **Phase 1:** the engine never auto-executes. Proposals appear as tickets; check the alerts/tickets UI for `CONFIRM ORDER [TICKET_ID]` strings.

7. Safety invariants (non-negotiable)

1. **No auto-execution in Phase 1.** Every engine proposal goes through `proposeTicket()` and waits for the exact `CONFIRM ORDER [TICKET_ID]` string, validated against the signed governance package.
2. **Timer full-reset.** Any 1-minute close below the zone floor resets the hold timer to 0 — never a pause. Implemented identically in live and backtest.
3. **Size from (entry – stop).** Quantity is derived from risk ($\leq 0.5\%$ of account) and capped by position value ($\leq 25\%$) — the most restrictive constraint wins, never conviction.
4. **OCO across variants and per day.** At most one confirmation per symbol per day; variant invalidation advances the ladder; all-invalidated marks `NO_TRADE`. Missing a trade is a valid outcome.
5. **Stops only tighten.** After the first target, the stop ratchets to breakeven — never widens.
6. **Governance clamping.** Configs exceeding constitutional ceilings are clamped down and the clamp recorded; without an active signed package the engine refuses to operate.
7. **Honest data.** Errors are surfaced verbatim; delayed data is flagged; no fabricated bars, ever.
8. **Same code path, three modes.** Live, backtest, and simulation share `SetupMachine.onBar()` — divergence between backtest and live results must be treated as a bug and flagged.

Appendix A — api/marketdata/indicators.ts

Indicator engine — pure functions: session VWAP, Wilder RSI(14), prior-day H/L/C, session H/L, RTH session grouping. — NEW FILE

```
/**
 * INDICATOR ENGINE — pure functions over 1-minute OHLCV bars.
 *
 * Everything the strategy engine needs per symbol per session:
 * - session VWAP (resets daily — computed only over today's RTH bars)
 * - RSI(14) on 1-minute closes (Wilder smoothing)
 * - prior day high / low / close
 * - session high / low
 *
 * Bars are filtered to US regular trading hours (09:30–16:00 America/New_York)
 * and grouped by ET calendar day. No external dependencies — the same module
 * is reused by the live feed, the backtester, and the what-if simulator so
 * all three modes compute identical values.
 */

export interface Bar {
  /** epoch ms */
  t: number;
  o: number;
  h: number;
  l: number;
  c: number;
  v: number;
}

export interface DayLevels {
  high: number;
  low: number;
  close: number;
}

export interface Indicators {
  symbol: string;
  asOf: number;
  barCount: number;
  last: number | null;
  vwap: number | null;
  rs14: number | null;
  sessionHigh: number | null;
  sessionLow: number | null;
  priorDay: DayLevels | null;
  sessionDate: string | null;
}

const ET_FORMATTER = new Intl.DateTimeFormat("en-CA", {
  timeZone: "America/New_York",
  year: "numeric",
  month: "2-digit",
  day: "2-digit",
  hour: "2-digit",
  minute: "2-digit",
  hourCycle: "h23",
});

/** ET calendar day (YYYY-MM-DD) and minutes-since-midnight for a timestamp. */
export function etParts(t: number): { day: string; minutes: number } {
  const parts = ET_FORMATTER.formatToParts(t);
  const get = (type: string) => parts.find((p) => p.type === type)?.value ?? "";
  return {
    day: `${get("year")}-${get("month")}-${get("day")}`,
    minutes: Number(get("hour")) * 60 + Number(get("minute")),
  };
}

/** Regular trading hours: bars starting 09:30 through 15:59 ET. */
const RTH_START = 9 * 60 + 30;
const RTH_END = 16 * 60;
```

```

/** Keep only regular-hours bars, grouped by ET session day, ascending. */
export function groupRthSessions(bars: Bar[]): Map<string, Bar[]> {
  const sessions = new Map<string, Bar[]>();
  for (const b of [...bars].sort((a, z) => a.t - z.t)) {
    const { day, minutes } = etParts(b.t);
    if (minutes < RTH_START || minutes >= RTH_END) continue;
    const list = sessions.get(day);
    if (list) list.push(b);
    else sessions.set(day, [b]);
  }
  return sessions;
}

/** Session VWAP:  $\Sigma(\text{typical} \times \text{vol}) / \Sigma \text{vol}$ , typical = (H+L+C)/3. Resets per session by construction. */
export function sessionVwap(bars: Bar[]): number | null {
  let pv = 0;
  let vol = 0;
  for (const b of bars) {
    const typical = (b.h + b.l + b.c) / 3;
    pv += typical * b.v;
    vol += b.v;
  }
  if (vol <= 0) return null;
  return +(pv / vol).toFixed(4);
}

/** Wilder's RSI on closes. Needs at least period+1 closes. */
export function rsi(closes: number[], period = 14): number | null {
  if (closes.length < period + 1) return null;
  let avgGain = 0;
  let avgLoss = 0;
  for (let i = 1; i <= period; i++) {
    const d = closes[i] - closes[i - 1];
    if (d >= 0) avgGain += d;
    else avgLoss -= d;
  }
  avgGain /= period;
  avgLoss /= period;
  for (let i = period + 1; i < closes.length; i++) {
    const d = closes[i] - closes[i - 1];
    avgGain = (avgGain * (period - 1) + Math.max(d, 0)) / period;
    avgLoss = (avgLoss * (period - 1) + Math.max(-d, 0)) / period;
  }
  if (avgLoss === 0) return 100;
  const rs = avgGain / avgLoss;
  return +(100 - 100 / (1 + rs)).toFixed(2);
}

export function highLow(bars: Bar[]): { high: number; low: number } | null {
  if (bars.length === 0) return null;
  let high = -Infinity;
  let low = Infinity;
  for (const b of bars) {
    if (b.h > high) high = b.h;
    if (b.l < low) low = b.l;
  }
  return { high: +high.toFixed(4), low: +low.toFixed(4) };
}

/**
 * Compute the full indicator set for one symbol from its raw 1-min bars.
 * The most recent session day is "the session"; the day before it supplies
 * prior-day levels.
 */
export function computeIndicators(symbol: string, bars: Bar[]): Indicators {
  const sessions = groupRthSessions(bars);
  const days = [...sessions.keys()].sort();
  const today = days.length > 0 ? sessions.get(days[days.length - 1])! : [];
  const prior = days.length > 1 ? sessions.get(days[days.length - 2])! : [];

  const hl = highLow(today);
  const priorHl = highLow(prior);

```

```

const lastBar = today.length > 0 ? today[today.length - 1] : null;

return {
  symbol,
  asOf: lastBar?.t ?? Date.now(),
  barCount: today.length,
  last: lastBar?.c ?? null,
  vwap: sessionVwap(today),
  rsi14: rsi(today.map((b) => b.c), 14),
  sessionHigh: hl?.high ?? null,
  sessionLow: hl?.low ?? null,
  priorDay: priorHl && prior.length > 0
    ? { high: priorHl.high, low: priorHl.low, close: prior[prior.length - 1].c }
    : null,
  sessionDate: days.length > 0 ? days[days.length - 1] : null,
};
}

```

Appendix B — api/marketdata/ibkr-data.ts

IBKR market data — Client Portal API: symbol → conid resolution, 1-min OHLCV history, gateway health. — NEW FILE

```
import type { Bar } from "./indicators";

/**
 * IBKR MARKET DATA — Client Portal Web API (CPAPI) market-data endpoints.
 *
 * Shares the gateway connection with the broker adapter (IBKR_GATEWAY_URL).
 * One IBKR gateway session serves both trading and market data — with a
 * funded account, 1-minute history and snapshots are real-time (subject to
 * the account's market-data subscriptions).
 *
 * Endpoints used:
 *   - GET /trsrv/stocks?symbols=...           symbol → conid resolution
 *   - GET /iserver/marketdata/history         1-min OHLCV bars
 *     ?conid=X&period=2d&bar=1min&outsideRth=false
 *   - GET /iserver/auth/status                gateway/session health
 *
 * Credentials note (same as the broker adapter): the operator authenticates
 * the Client Portal Gateway out-of-band; this module never logs in
 * programmatically. All failures surface as structured errors — the data
 * service degrades honestly rather than fabricating bars.
 */

const BASE = (process.env.IBKR_GATEWAY_URL ?? "https://localhost:5000/v1/api").replace(/\/$/, "");

const conidCache = new Map<string, number>();

async function req<T>(path: string): Promise<T> {
  const res = await fetch(`${BASE}${path}`, { method: "GET" });
  if (!res.ok) {
    const text = await res.text().catch(() => "");
    throw new Error(`IBKR marketdata GET ${path} → HTTP ${res.status} ${text.slice(0, 160)}`);
  }
  return (await res.json()) as T;
}

export interface GatewayHealth {
  ok: boolean;
  detail: string;
}

export async function gatewayHealth(): Promise<GatewayHealth> {
  try {
    const status = await req<{ authenticated?: boolean; connected?: boolean }>("/iserver/auth/status");
    if (status.authenticated) return { ok: true, detail: "gateway authenticated · market data live" };
    return { ok: false, detail: "gateway reachable but not authenticated — operator must log in to the Client Portal Gateway" };
  } catch (e) {
    return { ok: false, detail: (e as Error).message };
  }
}

/** Resolve a US stock symbol to an IBKR conid (cached). */
export async function resolveConid(symbol: string): Promise<number> {
  const key = symbol.toUpperCase();
  const hit = conidCache.get(key);
  if (hit) return hit;
  const rows = await req<Array<{ conid?: number }>>(`/trsrv/stocks?symbols=${encodeURIComponent(key)}`);
  const id = rows?.[0]?.conid;
  if (!id) throw new Error(`IBKR could not resolve symbol ${key} to a conid`);
  conidCache.set(key, id);
  return id;
}

interface IbkrHistoryBar {
  t?: number; // epoch ms
  o?: number;
  h?: number;
}
```

```

l?: number;
c?: number;
v?: number;
}

interface IbkrHistoryResponse {
  data?: IbkrHistoryBar[];
  points?: number;
  mktDataDelay?: number; // seconds of delay if data is delayed rather than real-time
}

export interface HistoryResult {
  symbol: string;
  conid: number;
  bars: Bar[];
  /** reported feed delay in seconds; 0/undefined = real-time */
  delaySeconds: number;
}

/**
 * Pull 1-minute OHLCV bars for a symbol. `period`/`bar` follow CPAPI
 * conventions ("1d"/"2d"/"1w", bar "1min"). outsideRth=false keeps the feed
 * regular-hours only, matching the indicator engine's RTH sessions.
 */
export async function fetchMinuteBars(symbol: string, period = "2d"): Promise<HistoryResult> {
  const conid = await resolveConid(symbol);
  const out = await req<IbkrHistoryResponse>(
    `/iserver/marketdata/history?conid=${conid}&period=${encodeURIComponent(period)}&bar=1min&outsideRth=false`,
  );
  const bars: Bar[] = (out.data ?? []).
    .filter((b) => typeof b.t === "number" && typeof b.c === "number")
    .map((b) => ({
      t: b.t as number,
      o: Number(b.o ?? b.c),
      h: Number(b.h ?? b.c),
      l: Number(b.l ?? b.c),
      c: Number(b.c),
      v: Number(b.v ?? 0),
    })))
    .sort((a, z) => a.t - z.t);
  return {
    symbol: symbol.toUpperCase(),
    conid,
    bars,
    delaySeconds: Number(out.mktDataDelay ?? 0),
  };
}

```

Appendix C — api/marketdata/service.ts

Market data service — watchlist, 60-second refresh loop (market hours only), per-symbol cache, honest error degradation. — NEW FILE

```
import { computeIndicators, etParts, type Bar, type Indicators } from "../indicators";
import { fetchMinuteBars, gatewayHealth } from "../ibkr-data";

/**
 * MARKET DATA SERVICE – the "Eyes" (module 0 of the strategy engine).
 *
 * Tracks a watchlist of symbols, pulls 1-minute bars from the IBKR gateway
 * every 60 seconds DURING REGULAR MARKET HOURS (09:30–16:00 ET, Mon–Fri),
 * and keeps a computed indicator set per symbol (session VWAP, RSI(14),
 * prior-day H/L/C, session H/L).
 *
 * Design rules:
 * - Honest degradation: if the gateway is down/unauthenticated, feeds carry
 *   the error and the UI shows it – no fabricated data, ever.
 * - The refresh loop only ticks during market hours; manual refreshes are
 *   allowed any time (e.g. for after-hours inspection of the last session).
 * - Delayed data is flagged: the feed reports the broker-reported delay so
 *   downstream timer logic can refuse to arm on stale bars.
 * - In-memory by design (module 0). Persistence arrives with the backtest
 *   module, which will also own historical bar storage.
 */

export interface SymbolFeed {
  symbol: string;
  tracked: boolean;
  bars: Bar[];
  indicators: Indicators | null;
  delaySeconds: number;
  lastRefresh: number | null;
  latencyMs: number | null;
  error: string | null;
}

export interface DataServiceStatus {
  provider: "IBKR";
  gatewayOk: boolean;
  gatewayDetail: string;
  marketOpen: boolean;
  loopRunning: boolean;
  refreshIntervalSeconds: number;
  symbols: Array<{
    symbol: string;
    ok: boolean;
    barCount: number;
    last: number | null;
    vwap: number | null;
    rs14: number | null;
    delaySeconds: number;
    lastRefresh: number | null;
    latencyMs: number | null;
    error: string | null;
  }>;
}

const REFRESH_MS = 60_000;

/** US regular session: Mon–Fri, 09:30–16:00 ET. */
export function isMarketHours(now = Date.now()): boolean {
  const { minutes } = etParts(now);
  const etDate = new Date(new Date(now).toLocaleString("en-US", { timeZone: "America/New_York" }));
  const dow = etDate.getDay();
  if (dow === 0 || dow === 6) return false;
  return minutes >= 9 * 60 + 30 && minutes < 16 * 60;
}

class MarketDataService {
  private feeds = new Map<string, SymbolFeed>();
}
```



```

private loop: ReturnType<typeof setInterval> | null = null;
private health: { ok: boolean; detail: string; checkedAt: number } | null = null;

async track(rawSymbol: string): Promise<SymbolFeed> {
  const symbol = rawSymbol.toUpperCase().trim();
  if (!/^[A-Z]{1,12}$/.test(symbol)) throw new Error(`Invalid symbol "${rawSymbol}"`);
  let feed = this.feeds.get(symbol);
  if (!feed) {
    feed = {
      symbol,
      tracked: true,
      bars: [],
      indicators: null,
      delaySeconds: 0,
      lastRefresh: null,
      latencyMs: null,
      error: null,
    };
    this.feeds.set(symbol, feed);
  }
  this.ensureLoop();
  await this.refresh(symbol);
  return feed;
}

untrack(rawSymbol: string): boolean {
  return this.feeds.delete(rawSymbol.toUpperCase().trim());
}

list(): SymbolFeed[] {
  return [...this.feeds.values()];
}

get(rawSymbol: string): SymbolFeed | undefined {
  return this.feeds.get(rawSymbol.toUpperCase().trim());
}

/** Refresh one symbol: pull bars, recompute indicators, record latency/errors. */
async refresh(rawSymbol: string): Promise<SymbolFeed> {
  const symbol = rawSymbol.toUpperCase().trim();
  const feed = this.feeds.get(symbol);
  if (!feed) throw new Error(`Symbol ${symbol} is not tracked`);
  const started = Date.now();
  try {
    const result = await fetchMinuteBars(symbol, "2d");
    feed.bars = result.bars;
    feed.indicators = computeIndicators(symbol, result.bars);
    feed.delaySeconds = result.delaySeconds;
    feed.error = result.bars.length === 0 ? "gateway returned no bars (market closed or no subscription)" :
null;
  } catch (e) {
    feed.error = (e as Error).message;
  }
  feed.latencyMs = Date.now() - started;
  feed.lastRefresh = Date.now();
  return feed;
}

async refreshAll(): Promise<void> {
  await Promise.allSettled([...this.feeds.keys()].map((s) => this.refresh(s)));
}

/** Start the 60-second loop if not already running. Ticks only in market hours. */
ensureLoop(): void {
  if (this.loop) return;
  this.loop = setInterval(() => {
    if (!isMarketHours()) return;
    void this.refreshAll();
  }, REFRESH_MS);
  // Never keep the process alive just for the data loop.
  if (typeof this.loop.unref === "function") this.loop.unref();
}

```

```

stopLoop(): void {
  if (this.loop) clearInterval(this.loop);
  this.loop = null;
}

async status(): Promise<DataServiceStatus> {
  // Re-check gateway health at most every 30 seconds.
  if (!this.health || Date.now() - this.health.checkedAt > 30_000) {
    const h = await gatewayHealth();
    this.health = { ...h, checkedAt: Date.now() };
  }
  return {
    provider: "IBKR",
    gatewayOk: this.health.ok,
    gatewayDetail: this.health.detail,
    marketOpen: isMarketHours(),
    loopRunning: this.loop !== null,
    refreshIntervalSeconds: REFRESH_MS / 1000,
    symbols: this.list().map((f) => ({
      symbol: f.symbol,
      ok: f.error === null && f.indicators !== null,
      barCount: f.indicators?.barCount ?? 0,
      last: f.indicators?.last ?? null,
      vwap: f.indicators?.vwap ?? null,
      rs14: f.indicators?.rs14 ?? null,
      delaySeconds: f.delaySeconds,
      lastRefresh: f.lastRefresh,
      latencyMs: f.latencyMs,
      error: f.error,
    })),
  };
}
}

/** Process-wide singleton. */
export const marketDataService = new MarketDataService();

```

Appendix D — api/marketdata-router.ts

Market data tRPC router — status / track / untrack / refresh / indicators / bars / list. — NEW FILE

```
import { z } from "zod";
import { authedQuery, createRouter } from "../middleware";
import { marketDataService } from "../marketdata/service";

/**
 * MARKET DATA API – module 0 of the autonomous strategy engine ("Eyes").
 *
 * Exposes the IBKR-backed data service: gateway/market-hours status,
 * watchlist management, cached 1-minute bars, and the computed indicator
 * set per symbol (session VWAP, RSI(14), prior-day H/L/C, session H/L).
 *
 * This is the minimum-disclosure surface: raw bar history is capped and
 * indicators are the primary output – downstream modules (triggers, state
 * machine) consume the same service server-side.
 */

const symbolInput = z.object({
  symbol: z.string().min(1).max(12),
});

export const marketDataRouter = createRouter({
  /** Service + gateway + per-symbol health. */
  status: authedQuery.query(async () => marketDataService.status()),

  /** Start tracking a symbol (immediate refresh + joins the 60-second loop). */
  track: authedQuery.input(symbolInput).mutation(async ({ input }) => {
    const feed = await marketDataService.track(input.symbol);
    return {
      symbol: feed.symbol,
      ok: feed.error === null,
      error: feed.error,
      barCount: feed.indicators?.barCount ?? 0,
      indicators: feed.indicators,
      delaySeconds: feed.delaySeconds,
      latencyMs: feed.latencyMs,
    };
  }),

  /** Stop tracking a symbol. */
  untrack: authedQuery.input(symbolInput).mutation(async ({ input }) => ({
    removed: marketDataService.untrack(input.symbol),
  })),

  /** Force a refresh of one tracked symbol (allowed outside market hours). */
  refresh: authedQuery.input(symbolInput).mutation(async ({ input }) => {
    const feed = await marketDataService.refresh(input.symbol);
    return { symbol: feed.symbol, ok: feed.error === null, error: feed.error, indicators: feed.indicators };
  }),

  /** Full indicator set for one tracked symbol. */
  indicators: authedQuery.input(symbolInput).query(async ({ input }) => {
    const feed = marketDataService.get(input.symbol);
    if (!feed) return { tracked: false as const, indicators: null, error: null, delaySeconds: 0 };
    return { tracked: true as const, indicators: feed.indicators, error: feed.error, delaySeconds:
feed.delaySeconds };
  }),

  /** Recent cached 1-minute bars (newest last), capped at `limit`. */
  bars: authedQuery
    .input(symbolInput.extend({ limit: z.number().int().min(1).max(390).default(120) }))
    .query(async ({ input }) => {
      const feed = marketDataService.get(input.symbol);
      if (!feed) return { tracked: false as const, bars: [] };
      return { tracked: true as const, bars: feed.bars.slice(-input.limit) };
    }),

  /** All tracked symbols with their indicator summary. */

```

```
list: authedQuery.query(async () => {  
  const status = await marketDataService.status();  
  return status.symbols;  
}),  
});
```

Appendix E — api/engine/triggers.ts

Trigger library (module 1) — 9 pure trigger functions, each returns armed + evidence. — NEW FILE

```
import { rsi, sessionVwap, type Bar, type DayLevels } from "../marketdata/indicators";

/**
 * TRIGGER LIBRARY — module 1 of the strategy engine.
 *
 * Every trigger is a standalone, PURE function over the session's 1-minute
 * bars. No hidden state: the same function called on the same bars returns
 * the same result — which is exactly why live mode and backtest mode agree.
 * Hold/timer logic (e.g. "30 consecutive minutes above VWAP") is derived
 * from the bar series itself: a close back below the floor makes the
 * trailing count zero — the full-reset rule falls out of the data.
 *
 * Each trigger returns { armed, evidence } where evidence is the data that
 * armed it (for logging and the audit trail).
 */

export interface TriggerContext {
  /** today's regular-hours bars, ascending, up to "now" */
  session: Bar[];
  priorDay: DayLevels | null;
  /** benchmark session bars (e.g. QQQ) for relative strength */
  benchmarkSession?: Bar[];
}

export type EvidenceValue = number | string | boolean | null;
export interface TriggerResult {
  armed: boolean;
  evidence: Record<string, EvidenceValue>;
}

export type TriggerParams = Record<string, number | string | boolean>;
export type TriggerFn = (ctx: TriggerContext, params: TriggerParams) => TriggerResult;

/* ----- helpers ----- */

/** Cumulative session VWAP at each bar index. */
export function rollingVwap(bars: Bar[]): number[] {
  let pv = 0;
  let vol = 0;
  return bars.map((b) => {
    pv += ((b.h + b.l + b.c) / 3) * b.v;
    vol += b.v;
    return vol > 0 ? pv / vol : b.c;
  });
}

/** Trailing count of consecutive closes at/above a floor series. */
function trailingHoldCount(bars: Bar[], floorSeries: number[]): number {
  let count = 0;
  for (let i = bars.length - 1; i >= 0; i--) {
    if (bars[i].c >= floorSeries[i]) count++;
    else break;
  }
  return count;
}

/** Local minima: bar low strictly below the `wing` neighbors on each side. */
function swingLows(bars: Bar[], wing = 2): Array<{ i: number; price: number }> {
  const out: Array<{ i: number; price: number }> = [];
  for (let i = wing; i < bars.length - wing; i++) {
    const l = bars[i].l;
    let isLow = true;
    for (let k = 1; k <= wing; k++) {
      if (bars[i - k].l <= l || bars[i + k].l <= l) { isLow = false; break; }
    }
    if (isLow) out.push({ i, price: l });
  }
}
```

```

    return out;
  }

  /** Wilder ATR over the session (falls back to whatever bars exist). */
  export function atr(bars: Bar[], period = 14): number | null {
    if (bars.length < 2) return null;
    const trs: number[] = [];
    for (let i = 1; i < bars.length; i++) {
      const prevC = bars[i - 1].c;
      const b = bars[i];
      trs.push(Math.max(b.h - b.l, Math.abs(b.h - prevC), Math.abs(b.l - prevC)));
    }
    const slice = trs.slice(-period);
    return slice.reduce((a, x) => a + x, 0) / slice.length;
  }

  /** Resolve a level token ("vwap" | "session_high" | "prior_day_high" | numeric string) to a number. */
  export function resolveLevel(ctx: TriggerContext, token: number | string): number | null {
    if (typeof token === "number") return token;
    const n = Number(token);
    if (Number.isFinite(n)) return n;
    const vw = sessionVwap(ctx.session);
    switch (token) {
      case "vwap": return vw;
      case "session_high": return ctx.session.length ? Math.max(...ctx.session.map((b) => b.h)) : null;
      case "session_low": return ctx.session.length ? Math.min(...ctx.session.map((b) => b.l)) : null;
      case "prior_day_high": return ctx.priorDay?.high ?? null;
      case "prior_day_low": return ctx.priorDay?.low ?? null;
      case "prior_day_close": return ctx.priorDay?.close ?? null;
      default: return null;
    }
  }

  /* ----- the triggers ----- */

  const vwapReclaim: TriggerFn = (ctx, p) => {
    const holdMinutes = Number(p.hold_minutes ?? 30);
    if (ctx.session.length < holdMinutes) return { armed: false, evidence: { reason: "insufficient_bars", have:
ctx.session.length, need: holdMinutes } };
    const vw = rollingVwap(ctx.session);
    const count = trailingHoldCount(ctx.session, vw);
    // "reclaim" requires price was below VWAP earlier in the session
    const wasBelow = ctx.session.some((b, i) => b.c < vw[i]);
    return {
      armed: wasBelow && count >= holdMinutes,
      evidence: { hold_count: count, hold_minutes: holdMinutes, was_below: wasBelow, vwap: vw[vw.length - 1], last:
ctx.session[ctx.session.length - 1].c },
    };
  };

  const pullbackHold: TriggerFn = (ctx, p) => {
    const holdMinutes = Number(p.hold_minutes ?? 10);
    const zoneLow = resolveLevel(ctx, p.zone_low as number | string);
    const zoneHigh = resolveLevel(ctx, p.zone_high as number | string);
    if (zoneLow === null || zoneHigh === null) return { armed: false, evidence: { reason: "unresolved_zone" } };
    const floorToken = (p.floor as number | string) ?? "vwap";
    const floorSeries = typeof floorToken === "string" && !Number.isFinite(Number(floorToken))
      ? (floorToken === "vwap" ? rollingVwap(ctx.session) : ctx.session.map(() => resolveLevel(ctx, floorToken) ??
Infinity))
      : ctx.session.map(() => Number(floorToken));
    const inZone = ctx.session.some((b) => b.l <= zoneHigh && b.h >= zoneLow);
    const count = trailingHoldCount(ctx.session, floorSeries);
    return {
      armed: inZone && count >= holdMinutes,
      evidence: { in_zone: inZone, hold_count: count, hold_minutes: holdMinutes, zone_low: zoneLow, zone_high:
zoneHigh, floor: String(floorToken) },
    };
  };

  const doubleBottom: TriggerFn = (ctx, p) => {
    const tolerancePct = Number(p.tolerance_pct ?? 1.0);
    const minApart = Number(p.min_minutes_apart ?? 20);
    const lows = swingLows(ctx.session);

```

```

if (lows.length < 2) return { armed: false, evidence: { swing_lows: lows.length } };
const second = lows[lows.length - 1];
const first = lows[lows.length - 2];
const gapBars = second.i - first.i;
const diffPct = (Math.abs(second.price - first.price) / Math.min(second.price, first.price)) * 100;
const last = ctx.session[ctx.session.length - 1];
const stillValid = last.l > second.price * 0.995; // second low not materially broken
const armed = diffPct <= tolerancePct && gapBars >= minApart && stillValid;
return {
  armed,
  evidence: { first_low: first.price, second_low: second.price, diff_pct: +diffPct.toFixed(2), minutes_apart:
gapBars, still_valid: stillValid },
};
};

const breakout: TriggerFn = (ctx, p) => {
  const level = resolveLevel(ctx, (p.level as number | string) ?? "session_high");
  const chaseLimitPct = Number(p.chase_limit_pct ?? 3.0);
  if (level === null || ctx.session.length === 0) return { armed: false, evidence: { reason: "unresolved_level" } };
  const last = ctx.session[ctx.session.length - 1].c;
  const chasePct = ((last - level) / level) * 100;
  const armed = last > level && chasePct <= chaseLimitPct;
  return { armed, evidence: { level, last, chase_pct: +chasePct.toFixed(2), chase_limit_pct: chaseLimitPct } };
};

const volumeClimax: TriggerFn = (ctx, p) => {
  const mult = Number(p.multiple_of_avg ?? 3.0);
  const lookback = Math.min(20, ctx.session.length - 1);
  if (lookback < 5) return { armed: false, evidence: { reason: "insufficient_bars" } };
  const last = ctx.session[ctx.session.length - 1];
  const avg = ctx.session.slice(-1 - lookback, -1).reduce((a, b) => a + b.v, 0) / lookback;
  const ratio = avg > 0 ? last.v / avg : 0;
  const followThrough = last.c >= last.o; // climax bar closes green
  const armed = ratio >= mult && followThrough;
  return { armed, evidence: { volume_ratio: +ratio.toFixed(2), multiple_required: mult, follow_through:
followThrough } };
};

const rsiDivergence: TriggerFn = (ctx, p) => {
  const rsiMax = Number(p.rsi_max ?? 35);
  const lows = swingLows(ctx.session);
  if (lows.length < 2) return { armed: false, evidence: { swing_lows: lows.length } };
  const second = lows[lows.length - 1];
  const first = lows[lows.length - 2];
  const rsiAt = (idx: number) => rsi(ctx.session.slice(0, idx + 1).map((b) => b.c), 14);
  const rsiFirst = rsiAt(first.i);
  const rsiSecond = rsiAt(second.i);
  const rsiNow = rsi(ctx.session.map((b) => b.c), 14);
  if (rsiFirst === null || rsiSecond === null || rsiNow === null) {
    return { armed: false, evidence: { reason: "insufficient_bars_for_rsi" } };
  }
  const lowerPriceLow = second.price < first.price;
  const higherRsiLow = rsiSecond > rsiFirst;
  const armed = lowerPriceLow && higherRsiLow && rsiNow < rsiMax;
  return {
    armed,
    evidence: { price_lows: `${first.price}-${second.price}`, rsi_lows: `${rsiFirst}-${rsiSecond}`, rsi_now:
rsiNow, rsi_max: rsiMax },
  };
};

const relativeStrength: TriggerFn = (ctx, p) => {
  const window = Number(p.window_minutes ?? 30);
  const bench = ctx.benchmarkSession;
  if (!bench || ctx.session.length < window || bench.length < window) {
    return { armed: false, evidence: { reason: "missing_benchmark_or_bars" } };
  }
  const symRet = (ctx.session[ctx.session.length - 1].c / ctx.session[ctx.session.length - window].c - 1) * 100;
  const benchRet = (bench[bench.length - 1].c / bench[bench.length - window].c - 1) * 100;
  const rs = symRet - benchRet;
  return { armed: rs > 0, evidence: { symbol_return_pct: +symRet.toFixed(2), benchmark_return_pct:
+benchRet.toFixed(2), rs_delta: +rs.toFixed(2), window_minutes: window } };
};

```

```

};

const priorDayHighReclaim: TriggerFn = (ctx) => {
  const pdh = ctx.priorDay?.high;
  if (!pdh || ctx.session.length === 0) return { armed: false, evidence: { reason: "no_prior_day" } };
  const last = ctx.session[ctx.session.length - 1].c;
  const wasBelow = ctx.session.some((b) => b.c < pdh);
  return { armed: wasBelow && last > pdh, evidence: { prior_day_high: pdh, last, was_below: wasBelow } };
};

const gapFill: TriggerFn = (ctx) => {
  const pdc = ctx.priorDay?.close;
  const open = ctx.session[0]?.o;
  if (!pdc || !open || ctx.session.length === 0) return { armed: false, evidence: { reason: "no_prior_day" } };
  const gappedDown = open < pdc;
  const last = ctx.session[ctx.session.length - 1].c;
  return { armed: gappedDown && last >= pdc, evidence: { prior_close: pdc, open, last, filled: last >= pdc } };
};

/* ----- registry ----- */

export const TRIGGERS: Record<string, { description: string; defaults: TriggerParams; evaluate: TriggerFn }> = {
  vwap_reclaim: { description: "Price reclaims VWAP and holds above it for N consecutive 1-min closes (resets on any close back below).", defaults: { hold_minutes: 30 }, evaluate: vwapReclaim },
  pullback_hold: { description: "Price pulls back into a defined zone and holds the floor for N consecutive closes (full reset on floor break).", defaults: { zone_low: "vwap", zone_high: "session_high", floor: "vwap", hold_minutes: 10 }, evaluate: pullbackHold },
  double_bottom: { description: "Two swing lows within tolerance % and at least N minutes apart; second low still intact.", defaults: { tolerance_pct: 1.0, min_minutes_apart: 20 }, evaluate: doubleBottom },
  breakout: { description: "Close above a level with price no more than chase-limit % past it.", defaults: { level: "session_high", chase_limit_pct: 3.0 }, evaluate: breakout },
  volume_climax: { description: "Last bar volume ≥ N× the 20-bar average, with green follow-through close.", defaults: { multiple_of_avg: 3.0 }, evaluate: volumeClimax },
  rsi_divergence: { description: "Lower price swing low but higher RSI low, with RSI now below the cap.", defaults: { rsi_max: 35 }, evaluate: rsiDivergence },
  relative_strength: { description: "Symbol outperforming its benchmark over the trailing window.", defaults: { window_minutes: 30 }, evaluate: relativeStrength },
  prior_day_high_reclaim: { description: "Close back above prior day high after trading below it this session.", defaults: {}, evaluate: priorDayHighReclaim },
  gap_fill: { description: "Down-gap fully filled back to prior close.", defaults: {}, evaluate: gapFill },
};

export function runTrigger(name: string, ctx: TriggerContext, params: TriggerParams = {}): TriggerResult {
  const trig = TRIGGERS[name];
  if (!trig) return { armed: false, evidence: { reason: `unknown_trigger:${name}` } };
  return trig.evaluate(ctx, { ...trig.defaults, ...params });
}

```


Appendix F — api/engine/config.ts

Strategy config system — config-as-data library + governance clamping against the signed package. — NEW FILE

```
import { loadActivePackage } from "../governance/runtime";
import type { TriggerParams } from "../triggers";

/**
 * STRATEGY CONFIG SYSTEM – the "library" (config-driven design).
 *
 * A strategy is DATA, not code: entry variants, stop rule, targets, hold
 * timer, sizing limits, schedule blocks. Adding a strategy = adding a
 * config. The engine (state machine, backtester, simulator, live monitor)
 * runs any config on any symbol the scanner feeds it.
 *
 * GOVERNANCE CLAMPING: every config is validated against the signed
 * governance package at load. Values that exceed constitutional ceilings
 * (e.g. risk limits from the compiled package) are CLAMPED DOWN and the
 * clamp is recorded – configs can tighten the constitution, never loosen
 * it.
 */

export interface EntryVariant {
  id: string;
  priority: number;
  trigger_type: string;
  params: TriggerParams;
  /** extra signal-stack requirement: at least N of `signals` armed */
  min_signals_armed: number;
  signals: string[];
  chase_limit_pct: number;
  /** BUY_STOP offset above the trigger price, in dollars */
  entry_offset: number;
}

export interface StrategyConfig {
  strategy_id: string;
  version: string;
  label: string;
  entry_variants: EntryVariant[];
  stop: { type: "vwap" | "swing_low" | "atr"; offset_atr?: number; offset_pct?: number };
  targets: Array<{ r_multiple: number; sell_fraction: number }>;
  hold_timer: { minutes: number; floor: "vwap" | number };
  sizing: { max_risk_pct: number; max_position_pct: number };
  schedule: { no_entries_after: string; flatten_by: string };
}

export interface ClampNote {
  field: string;
  requested: number;
  applied: number;
  reason: string;
}

export interface LoadedConfig {
  config: StrategyConfig;
  clamps: ClampNote[];
  governanceVersion: string;
}

/* ----- built-in generic configs (no tickers, no hardcoded trades) ----- */

export const BUILTIN_CONFIGS: StrategyConfig[] = [
  {
    strategy_id: "post_earnings_continuation",
    version: "1.0.0",
    label: "Post-Earnings Continuation",
    entry_variants: [
      { id: "A", priority: 1, trigger_type: "vwap_reclaim", params: { hold_minutes: 10 }, min_signals_armed: 2,
        signals: ["volume_climax", "relative_strength"], chase_limit_pct: 3.0, entry_offset: 0.05 },
      { id: "B", priority: 2, trigger_type: "breakout", params: { level: "session_high", chase_limit_pct: 3.0 },
```

```

min_signals_armed: 1, signals: ["volume_climax"], chase_limit_pct: 3.0, entry_offset: 0.05 },
  { id: "C", priority: 3, trigger_type: "prior_day_high_reclaim", params: {}, min_signals_armed: 1, signals:
["relative_strength"], chase_limit_pct: 3.0, entry_offset: 0.05 },
  ],
  stop: { type: "vwap", offset_pct: 0.1 },
  targets: [
    { r_multiple: 1.5, sell_fraction: 0.5 },
    { r_multiple: 3.0, sell_fraction: 0.3 },
    { r_multiple: 4.5, sell_fraction: 0.2 },
  ],
  hold_timer: { minutes: 10, floor: "vwap" },
  sizing: { max_risk_pct: 0.5, max_position_pct: 25 },
  schedule: { no_entries_after: "15:30", flatten_by: "15:55" },
},
{
  strategy_id: "bottom_fish_reversal",
  version: "1.0.0",
  label: "Down-Gap Bottom-Fish Reversal",
  entry_variants: [
    { id: "A", priority: 1, trigger_type: "double_bottom", params: { tolerance_pct: 1.0, min_minutes_apart: 20
}, min_signals_armed: 1, signals: ["rsi_divergence"], chase_limit_pct: 3.0, entry_offset: 0.05 },
    { id: "B", priority: 2, trigger_type: "vwap_reclaim", params: { hold_minutes: 30 }, min_signals_armed: 1,
signals: ["volume_climax"], chase_limit_pct: 3.0, entry_offset: 0.05 },
    { id: "C", priority: 3, trigger_type: "gap_fill", params: {}, min_signals_armed: 1, signals:
["rsi_divergence"], chase_limit_pct: 3.0, entry_offset: 0.05 },
  ],
  stop: { type: "swing_low", offset_atr: 0.5 },
  targets: [
    { r_multiple: 1.5, sell_fraction: 0.5 },
    { r_multiple: 3.0, sell_fraction: 0.5 },
  ],
  hold_timer: { minutes: 10, floor: "vwap" },
  sizing: { max_risk_pct: 0.5, max_position_pct: 25 },
  schedule: { no_entries_after: "15:30", flatten_by: "15:55" },
},
{
  strategy_id: "general_intraday_momentum",
  version: "1.0.0",
  label: "General Intraday Momentum",
  entry_variants: [
    { id: "A", priority: 1, trigger_type: "pullback_hold", params: { zone_low: "vwap", zone_high:
"session_high", floor: "vwap", hold_minutes: 10 }, min_signals_armed: 2, signals: ["relative_strength",
"volume_climax"], chase_limit_pct: 3.0, entry_offset: 0.05 },
    { id: "B", priority: 2, trigger_type: "breakout", params: { level: "prior_day_high", chase_limit_pct: 3.0
}, min_signals_armed: 1, signals: ["volume_climax"], chase_limit_pct: 3.0, entry_offset: 0.05 },
  ],
  stop: { type: "atr", offset_atr: 1.0 },
  targets: [
    { r_multiple: 1.0, sell_fraction: 0.5 },
    { r_multiple: 2.0, sell_fraction: 0.5 },
  ],
  hold_timer: { minutes: 10, floor: "vwap" },
  sizing: { max_risk_pct: 0.5, max_position_pct: 25 },
  schedule: { no_entries_after: "15:30", flatten_by: "15:55" },
},
];

export function getConfig(strategyId: string): StrategyConfig | undefined {
  return BUILTIN_CONFIGS.find((c) => c.strategy_id === strategyId);
}

/* ----- governance clamping ----- */

function packageValue(artifact: Record<string, Record<string, { value: unknown }>>, fullKey: string): unknown {
  for (const pack of Object.values(artifact)) {
    if (pack[fullKey]) return pack[fullKey].value;
  }
  return undefined;
}

/**
 * Validate + clamp a config against the signed governance package.
 * Never throws for over-risk – clamps down and records. Throws only if

```

```

* governance itself is unavailable (the constitution is load-bearing).
*/
export async function loadConfig(strategyId: string): Promise<LoadedConfig> {
  const found = getConfig(strategyId);
  if (!found) throw new Error(`Unknown strategy config "${strategyId}"`);
  const config: StrategyConfig = JSON.parse(JSON.stringify(found)) as StrategyConfig;
  const clamps: ClampNote[] = [];

  const pkg = await loadActivePackage(); // throws GOVERNANCE_UNAVAILABLE if none — by design
  const rawLimits = packageValue(pkg.artifact, "sizing.riskLimitsPct");
  let ceiling = 0.5;
  if (rawLimits && typeof rawLimits === "object") {
    const initial = (rawLimits as { initial?: string }).initial;
    const parsed = initial ? Number(initial.replace("%", "")) : NaN;
    if (Number.isFinite(parsed) && parsed > 0) ceiling = parsed;
  }
  if (config.sizing.max_risk_pct > ceiling) {
    clamps.push({
      field: "sizing.max_risk_pct",
      requested: config.sizing.max_risk_pct,
      applied: ceiling,
      reason: `clamped to constitutional ceiling ${ceiling}% (signed package ${pkg.version})`,
    });
    config.sizing.max_risk_pct = ceiling;
  }

  // Rescan cadence ceiling: hold timer may not exceed the intraday rescan ceiling.
  const cadence = packageValue(pkg.artifact, "rescan.cadence.intradayMinutes");
  const cadenceCeiling = Array.isArray(cadence) ? Math.max...(cadence as number[]) : typeof cadence ===
"number" ? cadence : null;
  if (cadenceCeiling !== null && config.hold_timer.minutes > cadenceCeiling * 3) {
    clamps.push({
      field: "hold_timer.minutes",
      requested: config.hold_timer.minutes,
      applied: cadenceCeiling * 3,
      reason: `clamped to 3× intraday rescan ceiling (${cadenceCeiling} min)`,
    });
    config.hold_timer.minutes = cadenceCeiling * 3;
  }

  return { config, clamps, governanceVersion: pkg.version };
}

```

Appendix G — api/engine/state-machine.ts

Setup state machine (module 2) — IDLE → WATCHING → CONFIRMED → IN_POSITION → OUT/KILLED, timer full-reset. — NEW FILE

```
import { etParts, sessionVwap, type Bar } from "../marketdata/indicators";
import { atr, rollingVwap, runTrigger, type EvidenceValue, type TriggerContext } from "../triggers";
import type { EntryVariant, StrategyConfig } from "../config";

/**
 * SETUP STATE MACHINE – module 2, the "Brain".
 *
 * IDLE → WATCHING → CONFIRMED → IN_POSITION → OUT
 *      └──────────┘ NO_TRADE      └──────────┘ KILLED (kill switch)
 *
 * Core invariants (from the spec, non-negotiable):
 * - WATCHING starts when price closes at/above the zone floor.
 * - Timer = consecutive 1-min closes above the floor. ANY close below
 *   the floor resets the timer to 0 – full reset, never a pause.
 * - Timer ≥ threshold AND variant trigger armed AND signal stack met
 *   → CONFIRMED → exactly one proposal (buy-stop entry).
 * - Variants are a ranked ladder: chase-limit breach invalidates the
 *   current variant and moves to the next. All invalidated → NO_TRADE.
 *   Missing a trade is a valid outcome.
 * - At most ONE confirmation per symbol per day (OCO across variants).
 *
 * The SAME machine runs live and in the backtester: live, a broker fills
 * the entry stop; in backtest, the replay engine calls notifyFill(). The
 * machine only ever sees bars up to "now" – no lookahead by construction.
 */

export type SetupState = "IDLE" | "WATCHING" | "CONFIRMED" | "IN_POSITION" | "OUT" | "KILLED" | "NO_TRADE";

export interface MachineEvent {
  t: number;
  type: "STATE_CHANGE" | "TIMER_RESET" | "CONFIRMED" | "FILL" | "EXIT" | "INVALIDATED" | "NO_TRADE" | "KILLED" |
  "BLOCKED";
  detail: string;
  data?: Record<string, EvidenceValue>;
}

export interface Proposal {
  symbol: string;
  strategyId: string;
  variantId: string;
  t: number;
  entry: number;
  stop: number;
  targets: Array<{ price: number; sellFraction: number }>;
  signalsArmed: string[];
  triggerEvidence: Record<string, EvidenceValue>;
}

interface Position {
  entry: number;
  stop: number;
  targets: Array<{ price: number; sellFraction: number; hit: boolean }>;
  filledAt: number;
}

function minutesOf(t: number): number {
  return etParts(t).minutes;
}

function hhmmToMinutes(hhmm: string): number {
  const [h, m] = hhmm.split(":").map(Number);
  return h * 60 + m;
}

export class SetupMachine {
  readonly symbol: string;
```

```

private config: StrategyConfig;
private events: MachineEvent[] = [];
private emit: (e: MachineEvent) => void;

state: SetupState = "IDLE";
timerCount = 0;
private variantIdx = 0;
private proposal: Proposal | null = null;
private position: Position | null = null;
exits: Array<{ t: number; price: number; fraction: number; reason: string }> = [];

constructor(symbol: string, config: StrategyConfig, onEvent?: (e: MachineEvent) => void) {
  this.symbol = symbol;
  this.config = config;
  this.emit = onEvent ?? (() => undefined);
}

private transition(to: SetupState, detail: string, data?: Record<string, EvidenceValue>): void {
  if (this.state === to) return;
  const e: MachineEvent = { t: Date.now(), type: "STATE_CHANGE", detail: `${this.state} → ${to}: ${detail}`,
data };
  this.state = to;
  this.events.push(e);
  this.emit(e);
}

private log(type: MachineEvent["type"], detail: string, data?: Record<string, EvidenceValue>): void {
  const e: MachineEvent = { t: Date.now(), type, detail, data };
  this.events.push(e);
  this.emit(e);
}

getEvents(): MachineEvent[] {
  return this.events;
}

get currentVariant(): EntryVariant | null {
  return this.config.entry_variants[this.variantIdx] ?? null;
}

/** Resolve the floor series for the hold timer ("vwap" or a fixed level). */
private floorSeries(ctx: TriggerContext): number[] {
  const floor = this.config.hold_timer.floor;
  if (floor === "vwap") return rollingVwap(ctx.session);
  return ctx.session.map(() => floor);
}

private resolveStop(ctx: TriggerContext, entry: number): number | null {
  const s = this.config.stop;
  const a = atr(ctx.session) ?? entry * 0.005;
  const last = ctx.session[ctx.session.length - 1];
  switch (s.type) {
    case "vwap": {
      const vw = sessionVwap(ctx.session);
      return vw === null ? null : +(vw * (1 - (s.offset_pct ?? 0) / 100)).toFixed(4);
    }
    case "swing_low": {
      const recent = Math.min(...ctx.session.slice(-20).map((b) => b.l));
      return +(recent - (s.offset_atr ?? 0.5) * a).toFixed(4);
    }
    case "atr":
    default:
      return +(last.c - (s.offset_atr ?? 1.0) * a).toFixed(4);
  }
}

/**
 * Feed one bar. `ctx` must contain the session UP TO AND INCLUDING this bar.
 * Returns a Proposal exactly once, on the IDLE/WATCHING → CONFIRMED transition.
 */
onBar(ctx: TriggerContext): Proposal | null {
  const bar = ctx.session[ctx.session.length - 1];
  if (!bar) return null;

```

```

const mins = minutesOf(bar.t);

if (this.state === "KILLED" || this.state === "NO_TRADE" || this.state === "OUT") return null;

// — position management (IN_POSITION) —
if (this.state === "IN_POSITION" && this.position) {
  this.managePosition(bar, mins);
  return null;
}

// — schedule gate: no new entries after the cutoff —
if (mins >= hmmmToMinutes(this.config.schedule.no_entries_after)) {
  if (this.state === "WATCHING") this.transition("IDLE", "past entry cutoff – stood down");
  return null;
}

const variant = this.currentVariant;
if (!variant) {
  if (this.state !== "NO_TRADE") {
    this.transition("NO_TRADE", "all entry variants invalidated – missing the trade is a valid outcome");
    this.log("NO_TRADE", "no viable variant remains");
  }
  return null;
}

// — hold timer: consecutive closes at/above floor, FULL reset below —
const floors = this.floorSeries(ctx);
const floorNow = floors[floors.length - 1];
if (bar.c >= floorNow) {
  this.timerCount += 1;
  if (this.state === "IDLE") this.transition("WATCHING", `price closed above floor (${floorNow.toFixed(2)}) – timer started`);
} else {
  if (this.timerCount > 0) this.log("TIMER_RESET", `close ${bar.c.toFixed(2)} below floor ${floorNow.toFixed(2)} – timer reset ${this.timerCount} → 0`);
  this.timerCount = 0;
  if (this.state === "WATCHING") this.transition("IDLE", "floor lost – back to scanning");
}

// — chase-limit invalidation → next variant (OCO ladder) —
const ref = ctx.session[0]?.o ?? bar.c;
const runPct = ((bar.c - ref) / ref) * 100;
if (this.timerCount > 0 && runPct > variant.chase_limit_pct + 3) {
  this.log("INVALIDATED", `variant ${variant.id} invalidated – price ran ${runPct.toFixed(1)}% past reference, chase limit ${variant.chase_limit_pct}%`);
  this.variantIdx += 1;
  this.timerCount = 0;
  this.transition("IDLE", `moving to next variant in ladder`);
  return null;
}

if (this.timerCount < this.config.hold_timer.minutes) return null;

// — timer threshold met: trigger + signal stack must also arm —
const trig = runTrigger(variant.trigger_type, ctx, variant.params);
if (!trig.armed) return null;

const signalsArmed = variant.signals.filter((name) => runTrigger(name, ctx).armed);
if (signalsArmed.length < variant.min_signals_armed) return null;

const entry = +(bar.c + variant.entry_offset).toFixed(4);
const stop = this.resolveStop(ctx, entry);
if (stop === null || stop >= entry) return null;

this.proposal = {
  symbol: this.symbol,
  strategyId: this.config.strategy_id,
  variantId: variant.id,
  t: bar.t,
  entry,
  stop,
  targets: this.config.targets.map((tg) => ({
    price: +(entry + tg.r_multiple * (entry - stop)).toFixed(4),

```

```

        sellFraction: tg.sell_fraction,
    })),
    signalsArmed,
    triggerEvidence: trig.evidence,
};
this.transition("CONFIRMED", `timer ${this.timerCount} ≥ ${this.config.hold_timer.minutes} · trigger
${variant.trigger_type} armed · signals [${signalsArmed.join(", ")}]`);
this.log("CONFIRMED", `proposal: entry ${entry} stop ${stop} targets ${this.proposal.targets.map((x) =>
x.price).join("/")}` , { entry, stop });
return this.proposal;
}

/** Called by the fill source (backtest replay or broker fill event). */
notifyFill(price: number, t: number): void {
    if (this.state !== "CONFIRMED" || !this.proposal) return;
    this.position = {
        entry: price,
        stop: this.proposal.stop,
        targets: this.proposal.targets.map((x) => ({ ...x, hit: false })),
        filledAt: t,
    };
    this.transition("IN_POSITION", `filled at ${price}`);
    this.log("FILL", `entry fill ${price} · protective stop ${this.position.stop} placed immediately`, { price,
stop: this.position.stop });
}

private managePosition(bar: Bar, mins: number): void {
    const pos = this.position;
    if (!pos) return;

    // flatten-by time: market exit
    if (mins >= hhmmToMinutes(this.config.schedule.flatten_by)) {
        this.exits.push({ t: bar.t, price: bar.c, fraction: 1, reason: "time_flatten" });
        this.log("EXIT", `time-based flatten at ${bar.c.toFixed(2)}`);
        this.transition("OUT", "flattened by schedule");
        return;
    }

    // stop: fills if the bar trades through it (checked before targets – conservative)
    if (bar.l <= pos.stop) {
        this.exits.push({ t: bar.t, price: pos.stop, fraction: 1, reason: "stop" });
        this.log("EXIT", `stop hit at ${pos.stop}`);
        this.transition("OUT", "protective stop filled");
        return;
    }

    // targets in order
    let remaining = 1 - this.exits.filter((e) => e.reason === "target").reduce((a, e) => a + e.fraction, 0);
    for (const tg of pos.targets) {
        if (!tg.hit && bar.h >= tg.price && remaining > 0) {
            tg.hit = true;
            const frac = Math.min(tg.sellFraction, remaining);
            this.exits.push({ t: bar.t, price: tg.price, fraction: frac, reason: "target" });
            this.log("EXIT", `target ${tg.price} filled (${(frac * 100).toFixed(0)}% of position)`);
            remaining -= frac;
            // ratchet stop to breakeven after first target – tightening only
            if (pos.stop < pos.entry) {
                pos.stop = pos.entry;
                this.log("EXIT", `stop tightened to breakeven ${pos.entry} (tightening-only rule)`);
            }
        }
    }
    if (remaining <= 0.0001) this.transition("OUT", "all targets filled");
}

kill(reason: string): void {
    if (this.state === "OUT" || this.state === "KILLED") return;
    this.transition("KILLED", reason);
    this.log("KILLED", reason);
}
}

```

Appendix H — api/engine/risk.ts

Risk engine — size from (entry-stop), ≤0.5% risk, ≤25% position cap, daily-loss halt, chase-cancel. — NEW FILE

```

/**
 * RISK ENGINE — position sizing (module 6 core, shared by live + backtest).
 *
 * Rules (constitutional):
 * - Share size comes FROM (entry - stop), never the other way around:
 *   qty = (equity × risk%) / (entry - stop)
 * - Per-trade risk ≤ max_risk_pct of account (0.5% ceiling via governance)
 * - Single position ≤ max_position_pct of account (25%)
 * - The most restrictive constraint wins — never conviction.
 */

export interface SizeResult {
  qty: number;
  perShareRisk: number;
  dollarRisk: number;
  positionValue: number;
  cappedBy: "risk" | "position_cap" | "zero";
}

export function sizePosition(input: {
  accountEquity: number;
  entry: number;
  stop: number;
  maxRiskPct: number;
  maxPositionPct: number;
}): SizeResult {
  const { accountEquity, entry, stop, maxRiskPct, maxPositionPct } = input;
  const perShareRisk = entry - stop;
  if (perShareRisk <= 0 || accountEquity <= 0) {
    return { qty: 0, perShareRisk: Math.max(perShareRisk, 0), dollarRisk: 0, positionValue: 0, cappedBy: "zero" };
  }
  const riskDollars = accountEquity * (maxRiskPct / 100);
  const qtyByRisk = riskDollars / perShareRisk;
  const capDollars = accountEquity * (maxPositionPct / 100);
  const qtyByCap = capDollars / entry;

  const cappedBy = qtyByRisk <= qtyByCap ? "risk" : "position_cap";
  const qty = Math.floor(Math.min(qtyByRisk, qtyByCap));
  return {
    qty,
    perShareRisk: +perShareRisk.toFixed(4),
    dollarRisk: +(qty * perShareRisk).toFixed(2),
    positionValue: +(qty * entry).toFixed(2),
    cappedBy: qty === 0 ? "zero" : cappedBy,
  };
}

/** Daily loss-limit check: halt new proposals at -3% (constitutional). */
export function dailyLossHalt(realizedPnlToday: number, accountEquity: number, limitPct = 3): boolean {
  if (accountEquity <= 0) return true;
  return realizedPnlToday <= -(accountEquity * (limitPct / 100));
}

/** Chase-cancel: if price ran more than `chaseLimitPct` past trigger before fill, cancel. */
export function chaseExceeded(triggerPrice: number, currentPrice: number, chaseLimitPct = 3): boolean {
  return ((currentPrice - triggerPrice) / triggerPrice) * 100 > chaseLimitPct;
}

```


Appendix I — api/engine/backtest.ts

Backtester (module 3) — bar-by-bar replay through the same machine, no lookahead, full report. — NEW FILE

```
import { groupRthSessions, type Bar, type DayLevels } from "../marketdata/indicators";
import type { StrategyConfig } from "../config";
import { SetupMachine, type MachineEvent, type Proposal } from "../state-machine";
import { sizePosition } from "../risk";

/**
 * BACKTEST MODULE — module 3.
 *
 * Replays 1-minute bars day-by-day through the SAME SetupMachine the live
 * monitor uses. No lookahead, by construction:
 * - the machine only ever receives bars up to the current index;
 * - a CONFIRMED proposal's buy-stop is evaluated for a fill on the NEXT
 *   bar (fills at the stop price, or the open if gapped through);
 * - stops fill before targets within a bar (conservative).
 *
 * Report: per-trade log, win rate, profit factor, expectancy (R), max
 * drawdown on the cumulative-R curve.
 */

export interface BacktestTrade {
  day: string;
  variantId: string;
  entryTime: number;
  exitTime: number;
  entry: number;
  avgExit: number;
  qty: number;
  stop: number;
  pnl: number;
  r: number;
  exitReason: string;
}

export interface BacktestReport {
  strategyId: string;
  symbol: string;
  days: number;
  trades: BacktestTrade[];
  totalTrades: number;
  wins: number;
  losses: number;
  winRate: number | null;
  profitFactor: number | null;
  expectancyR: number | null;
  maxDrawdownR: number;
  totalPnl: number;
  noTradeDays: number;
  events: MachineEvent[];
}

interface SessionDay {
  date: string;
  bars: Bar[];
  priorDay: DayLevels | null;
}

export function sessionsFromBars(bars: Bar[]): SessionDay[] {
  const grouped = groupRthSessions(bars);
  const days = [...grouped.keys()].sort();
  return days.map((date, i) => {
    const dayBars = grouped.get(date)!;
    const prior = i > 0 ? grouped.get(days[i - 1])! : null;
    return {
      date,
      bars: dayBars,
      priorDay: prior
      ? { high: Math.max(...prior.map((b) => b.h)), low: Math.min(...prior.map((b) => b.l)), close:

```

```

prior[prior.length - 1].c }
    : null,
  };
});
}

export function runBacktest(input: {
  config: StrategyConfig;
  symbol: string;
  bars: Bar[];
  accountEquity?: number;
  variantOnly?: string | null;
}): BacktestReport {
  const equity = input.accountEquity ?? 100000;
  const days = sessionsFromBars(input.bars);
  const trades: BacktestTrade[] = [];
  const allEvents: MachineEvent[] = [];
  let noTradeDays = 0;

  const config: StrategyConfig = input.variantOnly
    ? { ...input.config, entry_variants: input.config.entry_variants.filter((v) => v.id === input.variantOnly) }
    : input.config;

  for (const day of days) {
    const machine = new SetupMachine(input.symbol, config, (e) => allEvents.push(e));
    let pending: Proposal | null = null;
    let pendingQty = 0;

    for (let i = 0; i < day.bars.length; i++) {
      const bar = day.bars[i];

      // 1) fill check for a pending entry stop – BEFORE feeding this bar's close
      if (pending && machine.state === "CONFIRMED") {
        const chasePct = ((bar.o - pending.entry) / pending.entry) * 100;
        if (chasePct > config.entry_variants.find((v) => v.id === pending!.variantId)?.chase_limit_pct!) {
          machine.kill(`chase-cancel: open gapped ${chasePct.toFixed(1)}% past trigger`);
          pending = null;
        } else if (bar.h >= pending.entry) {
          const fill = Math.max(pending.entry, bar.o);
          machine.notifyFill(fill, bar.t);
          pendingQty = sizePosition({
            accountEquity: equity,
            entry: fill,
            stop: pending.stop,
            maxRiskPct: config.sizing.max_risk_pct,
            maxPositionPct: config.sizing.max_position_pct,
          }).qty;
          pending = { ...pending, entry: fill };
        }
      }

      // 2) feed the bar (machine sees only up-to-now)
      const ctx = { session: day.bars.slice(0, i + 1), priorDay: day.priorDay };
      const proposal = machine.onBar(ctx);
      if (proposal) pending = proposal;
    }

    // 3) settle the day
    if (machine.state === "IN_POSITION" || machine.exits.length > 0) {
      const exits = machine.exits;
      if (machine.state === "IN_POSITION") {
        // machine flattens at 15:55 by schedule; if bars ended early, close at last bar
        const last = day.bars[day.bars.length - 1];
        exits.push({ t: last.t, price: last.c, fraction: 1, reason: "eod" });
      }
      const pos = (machine as unknown as { proposal: Proposal | null }).proposal;
      if (pos && pendingQty > 0 && exits.length > 0) {
        let remaining = 1;
        let weighted = 0;
        for (const ex of exits) {
          const f = Math.min(ex.fraction, remaining);
          weighted += f * ex.price;
          remaining -= f;
        }
      }
    }
  }
}

```

```

    }
    if (remaining > 0.0001) weighted += remaining * day.bars[day.bars.length - 1].c;
    const perShareRisk = pos.entry - pos.stop;
    const pnl = (weighted - pos.entry) * pendingQty;
    trades.push({
      day: day.date,
      variantId: pos.variantId,
      entryTime: pos.t,
      exitTime: exits[exits.length - 1].t,
      entry: pos.entry,
      avgExit: +weighted.toFixed(4),
      qty: pendingQty,
      stop: pos.stop,
      pnl: +pnl.toFixed(2),
      r: perShareRisk > 0 ? +((weighted - pos.entry) / perShareRisk).toFixed(2) : 0,
      exitReason: exits[exits.length - 1].reason,
    });
  }
} else if (machine.state === "NO_TRADE" || machine.state === "IDLE" || machine.state === "WATCHING") {
  noTradeDays += 1;
}
}

const wins = trades.filter((t) => t.pnl > 0);
const losses = trades.filter((t) => t.pnl <= 0);
const grossWin = wins.reduce((a, t) => a + t.pnl, 0);
const grossLoss = Math.abs(losses.reduce((a, t) => a + t.pnl, 0));
let cum = 0;
let peak = 0;
let maxDd = 0;
for (const t of trades) {
  cum += t.r;
  peak = Math.max(peak, cum);
  maxDd = Math.max(maxDd, peak - cum);
}

return {
  strategyId: config.strategy_id,
  symbol: input.symbol,
  days: days.length,
  trades,
  totalTrades: trades.length,
  wins: wins.length,
  losses: losses.length,
  winRate: trades.length ? +((wins.length / trades.length) * 100).toFixed(1) : null,
  profitFactor: grossLoss > 0 ? +(grossWin / grossLoss).toFixed(2) : null,
  expectancyR: trades.length ? +(trades.reduce((a, t) => a + t.r, 0) / trades.length).toFixed(3) : null,
  maxDrawdownR: +maxDd.toFixed(2),
  totalPnl: +trades.reduce((a, t) => a + t.pnl, 0).toFixed(2),
  noTradeDays,
  events: allEvents.slice(-200),
};
}

```

Appendix J — api/engine/simulator.ts

What-if simulator (module 4) — per-variant expectancy ranking, drops negative variants, re-ranks ladder. — NEW FILE

```
import type { Bar } from "../marketdata/indicators";
import type { StrategyConfig } from "../config";
import { runBacktest } from "../backtest";

/**
 * WHAT-IF SIMULATOR – module 4.
 *
 * Before going live on a candidate, every entry variant is simulated
 * against the available 1-minute history (same config, same machine, same
 * fill model – the backtester restricted to one variant at a time).
 * Variants are ranked by expectancy; negative-expectancy variants are
 * dropped. The resulting ranking IS the fallback ladder the live monitor
 * uses.
 */

export interface VariantSimResult {
  variantId: string;
  triggerType: string;
  trades: number;
  winRate: number | null;
  expectancyR: number | null;
  profitFactor: number | null;
  maxDrawdownR: number;
  totalPnl: number;
  kept: boolean;
  dropReason: string | null;
}

export interface SimulationReport {
  strategyId: string;
  symbol: string;
  sampleDays: number;
  variants: VariantSimResult[];
  rankedLadder: string[]; // variant ids in priority order after simulation
  recommendation: string;
}

export function simulateVariants(input: {
  config: StrategyConfig;
  symbol: string;
  bars: Bar[];
  minTrades?: number;
}): SimulationReport {
  const minTrades = input.minTrades ?? 2;
  const variants: VariantSimResult[] = input.config.entry_variants.map((v) => {
    const report = runBacktest({ config: input.config, symbol: input.symbol, bars: input.bars, variantOnly: v.id });
    const insufficient = report.totalTrades < minTrades;
    const negative = report.expectancyR !== null && report.expectancyR < 0;
    const kept = !insufficient && !negative;
    return {
      variantId: v.id,
      triggerType: v.trigger_type,
      trades: report.totalTrades,
      winRate: report.winRate,
      expectancyR: report.expectancyR,
      profitFactor: report.profitFactor,
      maxDrawdownR: report.maxDrawdownR,
      totalPnl: report.totalPnl,
      kept,
      dropReason: insufficient
        ? `insufficient sample (${report.totalTrades} trades < ${minTrades})`
        : negative
        ? `negative expectancy (${report.expectancyR}R)`
        : null,
    };
  });
}
```

```

    const kept = variants.filter((v) => v.kept).sort((a, b) => (b.expectancyR ?? -Infinity) - (a.expectancyR ?? -Infinity));
    const rankedLadder = [...kept.map((v) => v.variantId), ...variants.filter((v) => !v.kept).map((v) => v.variantId)];

    const best = kept[0];
    const recommendation = best
      ? `Lead with variant ${best.variantId} (${best.triggerType}) – expectancy ${best.expectancyR}R over ${best.trades} simulated trades. ${variants.length - kept.length} variant(s) dropped.`
      : "No variant survived simulation – stand down on this candidate. Missing the trade is a valid outcome.";

    return {
      strategyId: input.config.strategy_id,
      symbol: input.symbol,
      sampleDays: 0,
      variants,
      rankedLadder,
      recommendation,
    };
  }
}

```

Appendix K — api/engine/scanner.ts

Scanner hook (module 5) — gap %, RVOL, vs-VWAP candidates + best-fit config selection. — NEW FILE

```
import type { SymbolFeed } from "../marketdata/service";
import { computeIndicators } from "../marketdata/indicators";
import { BUILTIN_CONFIGS } from "../config";

/**
 * SCANNER HOOK — module 5.
 *
 * Scans the tracked watchlist feeds and emits candidates with metadata:
 * gap %, relative volume, position vs VWAP, and the best-fit strategy
 * config selected by simple, auditable rules:
 * - down-gap below VWAP → bottom-fish reversal config
 * - up-gap / strength > VWAP → continuation config
 * - everything else liquid → general intraday momentum
 *
 * A broad 500–2,000 symbol universe scan requires a market-wide data
 * source; this hook scans whatever the data service tracks (module 0)
 * and plugs into the same loop: SCAN → SIMULATE → RANK → MONITOR.
 */

export interface Candidate {
  symbol: string;
  last: number | null;
  gapPct: number | null;
  relativeVolume: number | null;
  vsVwapPct: number | null;
  rs14: number | null;
  suggestedConfig: string;
  suggestedLabel: string;
  reason: string;
}

const MIN_RVOL = 1.5;
const MIN_GAP_PCT = 2.0;

export function scanFeeds(feeds: SymbolFeed[]): Candidate[] {
  const out: Candidate[] = [];
  for (const f of feeds) {
    if (!f.indicators || f.bars.length < 30) continue;
    const ind = f.indicators;
    const gapPct = ind.priorDay.close > 0 && f.bars.length > 0
      ? +(((f.bars[0].o - ind.priorDay.close) / ind.priorDay.close) * 100).toFixed(2)
      : null;

    // RVOL: today's volume so far vs prior-day volume over the same bar count
    const todayVol = f.bars.slice(-ind.barCount).reduce((a, b) => a + b.v, 0);
    const prior = computeIndicators(f.symbol, f.bars.slice(0, Math.max(f.bars.length - ind.barCount, 0)));
    const priorVolSameWindow = prior.barCount > 0
      ? f.bars.slice(0, f.bars.length - ind.barCount).slice(-Math.min(ind.barCount, prior.barCount)).reduce((a, b) => a + b.v, 0)
      : 0;
    const relativeVolume = priorVolSameWindow > 0 ? +(todayVol / priorVolSameWindow).toFixed(2) : null;

    const vsVwapPct = ind.vwap && ind.last ? +(((ind.last - ind.vwap) / ind.vwap) * 100).toFixed(2) : null;

    const isGap = gapPct !== null && Math.abs(gapPct) >= MIN_GAP_PCT;
    const isHot = relativeVolume !== null && relativeVolume >= MIN_RVOL;
    if (!isGap && !isHot) continue;

    let suggestedConfig = "general_intraday_momentum";
    let reason = "liquid mover — default momentum config";
    if (gapPct !== null && gapPct <= -MIN_GAP_PCT && (vsVwapPct ?? 0) < 0) {
      suggestedConfig = "bottom_fish_reversal";
      reason = `down-gap ${gapPct}% below VWAP — reversal config`;
    } else if ((gapPct !== null && gapPct >= MIN_GAP_PCT) || (vsVwapPct ?? 0) > 0) {
      suggestedConfig = "post_earnings_continuation";
      reason = gapPct !== null && gapPct >= MIN_GAP_PCT ? `up-gap ${gapPct}% — continuation config` : "holding above VWAP — continuation config";
    }
  }
}
```

```
    }  
    const cfg = BUILTIN_CONFIGS.find((c) => c.strategy_id === suggestedConfig);  
    out.push({  
      symbol: f.symbol,  
      last: ind.last,  
      gapPct,  
      relativeVolume,  
      vsVwapPct,  
      rsi14: ind.rsi14,  
      suggestedConfig,  
      suggestedLabel: cfg?.label ?? suggestedConfig,  
      reason,  
    });  
  }  
  return out.sort((a, b) => (b.relativeVolume ?? 0) - (a.relativeVolume ?? 0));  
}
```

Appendix L — api/engine/monitor.ts

Engine monitor (module 6) — live loop wiring CONFIRMED setups into the ticket confirmation gate. — NEW FILE

```
import { groupRthSessions } from "../marketdata/indicators";
import { marketDataService } from "../marketdata/service";
import { loadConfig } from "../config";
import { SetupMachine, type MachineEvent, type Proposal } from "../state-machine";
import { proposeTicket } from "../queries/tickets";
import { sizePosition } from "../risk";
import type { PublicTicket } from "../queries/tickets";

/**
 * ENGINE MONITOR — module 6, execution wiring.
 *
 * The live autonomous loop for one user + symbol + strategy config:
 *
 * feed refresh (module 0) → SetupMachine evaluates latest bar
 * → CONFIRMED proposal → governance-clamped risk sizing
 * → proposeTicket(...) → READY_FOR_CONFIRMATION
 * → human types CONFIRM ORDER [TICKET_ID] → existing broker path
 *
 * Phase 1 by design: the engine NEVER auto-executes. Every proposal lands
 * in the same ticketed confirmation gate used by Intelligence — the exact
 * CONFIRM string validated against the signed governance package.
 * One proposal per symbol per day (OCO across variants is enforced by the
 * machine itself; the monitor adds the per-day idempotence).
 */

const PAPER_EQUITY = 100_000; // paper-mode account equity until a live broker account is bound

interface ActiveMonitor {
  userId: number;
  symbol: string;
  strategyId: string;
  machine: SetupMachine;
  lastProposalDay: string | null;
  lastEvents: MachineEvent[];
  startedAt: number;
}

const monitors = new Map<string, ActiveMonitor>();

function key(userId: number, symbol: string): string {
  return `${userId}:${symbol.toUpperCase()}`;
}

function etDay(t: number): string {
  return new Date(t).toLocaleDateString("en-CA", { timeZone: "America/New_York" });
}

export async function startMonitor(userId: number, symbol: string, strategyId: string): Promise<{ ok: boolean;
detail: string }> {
  const sym = symbol.toUpperCase();
  const { config, clamps, governanceVersion } = await loadConfig(strategyId); // throws GOVERNANCE_UNAVAILABLE —
  by design
  const machine = new SetupMachine(sym, config);
  monitors.set(key(userId, sym), {
    userId,
    symbol: sym,
    strategyId,
    machine,
    lastProposalDay: null,
    lastEvents: [],
    startedAt: Date.now(),
  });
  const clampNote = clamps.length > 0 ? ` · clamps: ${clamps.map((c) => `${c.field} ${c.requested}
→${c.applied}`).join(", ")}` : "";
  return { ok: true, detail: `monitoring ${sym} with ${config.label} under governance
${governanceVersion}${clampNote}` };
}
```



```

export function stopMonitor(userId: number, symbol: string): boolean {
  return monitors.delete(key(userId, symbol));
}

export function listMonitors(userId: number): Array<{ symbol: string; strategyId: string; state: string;
timerCount: number; startedAt: number; events: MachineEvent[] }> {
  return [...monitors.values()]
    .filter((m) => m.userId === userId)
    .map((m) => ({ symbol: m.symbol, strategyId: m.strategyId, state: m.machine.state, timerCount:
m.machine.timerCount, startedAt: m.startedAt, events: m.lastEvents.slice(-20) }));
}

export interface EvaluateResult {
  symbol: string;
  state: string;
  timerCount: number;
  proposal: Proposal | null;
  ticket: PublicTicket | null;
  detail: string;
}

/**
 * Evaluate one monitored symbol against its freshest bars. Called by the
 * router (manual evaluate) and intended to be invoked after each data
 * refresh tick.
 */
export async function evaluateMonitor(userId: number, symbol: string): Promise<EvaluateResult> {
  const sym = symbol.toUpperCase();
  const mon = monitors.get(key(userId, sym));
  if (!mon) return { symbol: sym, state: "NOT_MONITORED", timerCount: 0, proposal: null, ticket: null, detail:
"no active monitor – start one first" };

  const feed = marketDataService.get(sym);
  if (!feed || feed.bars.length === 0) {
    return { symbol: sym, state: mon.machine.state, timerCount: mon.machine.timerCount, proposal: null, ticket:
null, detail: feed?.error ?? "no bars – track the symbol in the data feed first" };
  }

  const sessions = groupRthSessions(feed.bars);
  const days = [...sessions.keys()].sort();
  const todayBars = sessions.get(days[days.length - 1]) ?? [];
  const prior = days.length > 1 ? sessions.get(days[days.length - 2])! : null;
  const priorDay = prior
    ? { high: Math.max(...prior.map((b) => b.h)), low: Math.min(...prior.map((b) => b.l)), close:
prior[prior.length - 1].c }
    : null;

  const events: MachineEvent[] = [];
  const proposal = mon.machine.onBar({ session: todayBars, priorDay });
  mon.lastEvents = mon.machine.getEvents().slice(-50);
  events.push(...mon.machine.getEvents().slice(-3));

  let ticket: PublicTicket | null = null;
  let detail = `state ${mon.machine.state} · timer ${mon.machine.timerCount}`;

  if (proposal) {
    const day = etDay(proposal.t);
    if (mon.lastProposalDay === day) {
      detail = "proposal suppressed – already proposed for this symbol today (OCO per-day rule)";
    } else {
      mon.lastProposalDay = day;
      const { config } = await loadConfig(mon.strategyId);
      const size = sizePosition({
        accountEquity: PAPER_EQUITY,
        entry: proposal.entry,
        stop: proposal.stop,
        maxRiskPct: config.sizing.max_risk_pct,
        maxPositionPct: config.sizing.max_position_pct,
      });
      if (size.qty <= 0) {
        detail = "proposal blocked by risk engine – size resolved to zero";
      } else {

```

```

    const result = await proposeTicket(userId, {
      strategy: proposal.strategyId.toUpperCase().replace(/^[A-Z]/g, "").slice(0, 6) || "ENGINE",
      broker: "PAPER",
      symbol: sym,
      side: "BUY",
      quantity: size.qty,
      orderType: "STP",
      stopPrice: proposal.entry,
      tif: "DAY",
      entry: proposal.entry,
      stop: proposal.stop,
      target: proposal.targets[0]?.price,
      origin: "AUTONOMOUS",
    });
    ticket = result.ticket;
    detail = `CONFIRMED → ticket ${result.ticket.ticketId} staged (${size.qty} sh, risk ${size.dollarRisk},
capped by ${size.cappedBy}) - awaiting CONFIRM ORDER ${result.ticket.ticketId}`;
  }
}

return { symbol: sym, state: mon.machine.state, timerCount: mon.machine.timerCount, proposal, ticket, detail };
}

/** Evaluate every active monitor (call after data-feed refresh ticks). */
export async function evaluateAll(): Promise<number> {
  let n = 0;
  for (const m of [...monitors.values()]) {
    await evaluateMonitor(m.userId, m.symbol).catch(() => undefined);
    n += 1;
  }
  return n;
}

```

Appendix M — api/engine-router.ts

Engine tRPC router — triggers / configs / backtest / simulate / scan / monitor lifecycle / evaluate. — NEW FILE

```
import { z } from "zod";
import { authedQuery, createRouter } from "../middleware";
import { TRIGGERS } from "../engine/triggers";
import { BUILTIN_CONFIGS, loadConfig } from "../engine/config";
import { runBacktest } from "../engine/backtest";
import { simulateVariants } from "../engine/simulator";
import { scanFeeds } from "../engine/scanner";
import { evaluateMonitor, listMonitors, startMonitor, stopMonitor } from "../engine/monitor";
import { marketDataService } from "../marketdata/service";
import { fetchMinuteBars } from "../marketdata/ibkr-data";

/**
 * ENGINE API — modules 1–6 of the autonomous strategy engine.
 *
 * Trigger library, config library (governance-clamped), backtester,
 * what-if simulator, scanner, and the live monitor that wires CONFIRMED
 * setups into the existing ticket confirmation gate. Phase 1: no
 * auto-execution — every proposal waits for CONFIRM ORDER [TICKET_ID].
 */

const strategyInput = z.object({
  symbol: z.string().min(1).max(12),
  strategyId: z.string().min(1).max(64),
});

/** Fetch bars for analysis: prefer the tracked feed, else pull fresh from IBKR. */
async function barsFor(symbol: string, period: string) {
  const feed = marketDataService.get(symbol);
  if (feed && feed.bars.length >= 100) return feed.bars;
  const result = await fetchMinuteBars(symbol, period);
  return result.bars;
}

export const engineRouter = createRouter({
  /** Trigger library catalog (module 1). */
  triggers: authedQuery.query(() =>
    Object.entries(TRIGGERS).map(([name, t]) => ({ name, description: t.description, defaults: t.defaults })),
  ),

  /** Strategy config library (minimum-disclosure summary). */
  configs: authedQuery.query(() =>
    BUILTIN_CONFIGS.map((c) => ({
      strategyId: c.strategy_id,
      label: c.label,
      version: c.version,
      variants: c.entry_variants.map((v) => ({ id: v.id, trigger: v.trigger_type, priority: v.priority })),
      riskPct: c.sizing.max_risk_pct,
      positionCapPct: c.sizing.max_position_pct,
    })),
  ),

  /** Backtest a config on a symbol's recent 1-min history (module 3). */
  backtest: authedQuery
    .input(strategyInput.extend({ period: z.enum(["2d", "1w", "2w", "1m"]).default("1w") }))
    .mutation(async ({ input }) => {
      const { config, clamps, governanceVersion } = await loadConfig(input.strategyId);
      const bars = await barsFor(input.symbol, input.period);
      if (bars.length < 200) throw new Error(`Not enough bars for ${input.symbol.toUpperCase()} (${bars.length})`);
      // — check the data feed
      const report = runBacktest({ config, symbol: input.symbol.toUpperCase(), bars });
      return { ...report, events: report.events.slice(-30), governanceVersion, clamps };
    }),

  /** What-if simulator: rank entry variants by expectancy (module 4). */
  simulate: authedQuery
    .input(strategyInput.extend({ period: z.enum(["2d", "1w", "2w", "1m"]).default("1w") }))
    .mutation(async ({ input }) => {
```

```

    const { config, clamps, governanceVersion } = await loadConfig(input.strategyId);
    const bars = await barsFor(input.symbol, input.period);
    if (bars.length < 200) throw new Error(`Not enough bars for ${input.symbol.toUpperCase()}
    (${bars.length})`);
    const report = simulateVariants({ config, symbol: input.symbol.toUpperCase(), bars });
    return { ...report, governanceVersion, clamps };
  }},

  /** Scan tracked feeds for candidates (module 5). */
  scan: authedQuery.mutation(async () => scanFeeds(marketDataService.list())),

  /** Start live monitoring: symbol x config → confirmation-gated proposals (module 6). */
  monitorStart: authedQuery.input(strategyInput).mutation(async ({ ctx, input }) => {
    await marketDataService.track(input.symbol); // ensure the feed exists
    return startMonitor(ctx.user.id, input.symbol, input.strategyId);
  })),

  monitorStop: authedQuery.input(z.object({ symbol: z.string().min(1).max(12) })).mutation(async ({ ctx, input })
=> ({
    stopped: stopMonitor(ctx.user.id, input.symbol),
  })),

  monitors: authedQuery.query(async ({ ctx }) => listMonitors(ctx.user.id)),

  /** Evaluate a monitored symbol against its freshest bars right now. */
  evaluate: authedQuery.input(z.object({ symbol: z.string().min(1).max(12) })).mutation(async ({ ctx, input }) =>
{
  await marketDataService.refresh(input.symbol).catch(() => undefined);
  return evaluateMonitor(ctx.user.id, input.symbol);
}),
});

```

Appendix N — src/components/DataFeedPanel.tsx

Live Data Feed panel (React) — gateway status, watchlist tracking, per-symbol indicators. — NEW FILE

```
import { useState } from 'react';
import { Activity, Plus, RefreshCw, Satellite, Trash2 } from 'lucide-react';
import { trpc } from '@providers/trpc';
import { cn } from '@lib/utls';

/**
 * LIVE DATA FEED PANEL – module 0 of the strategy engine ("Eyes").
 *
 * Shows the IBKR gateway connection, market-hours state, the 60-second
 * refresh loop, and per-symbol indicators (session VWAP, RSI-14, session
 * high/low, prior-day levels). Errors are displayed verbatim – a down or
 * unauthenticated gateway is visible here, never hidden.
 */
export default function DataFeedPanel() {
  const utls = trpc.useUtls();
  const [symbol, setSymbol] = useState('');
  const { data: status } = trpc.marketData.status.useQuery(undefined, { refetchInterval: 15000 });
  const { data: symbols } = trpc.marketData.list.useQuery(undefined, { refetchInterval: 15000 });

  const trackMut = trpc.marketData.track.useMutation({
    onSettled: () => {
      void utls.marketData.status.invalidate();
      void utls.marketData.list.invalidate();
    },
  });
  const untrackMut = trpc.marketData.untrack.useMutation({
    onSettled: () => {
      void utls.marketData.status.invalidate();
      void utls.marketData.list.invalidate();
    },
  });
  const refreshMut = trpc.marketData.refresh.useMutation({
    onSettled: () => {
      void utls.marketData.status.invalidate();
      void utls.marketData.list.invalidate();
    },
  });

  const submit = () => {
    const s = symbol.trim().toUpperCase();
    if (!s) return;
    trackMut.mutate({ symbol: s });
    setSymbol('');
  };

  const gatewayTone = status?.gatewayOk ? 'text-emerald-600' : 'text-rose-600';

  return (
    <section className="glass rounded-2xl p-6">
      <div className="mb-4 flex flex-wrap items-center justify-between gap-3">
        <div className="flex items-start gap-3">
          <div className="grid h-9 w-9 shrink-0 place-items-center rounded-xl bg-gradient-to-br from-royal-500 to-teal-500 text-white">
            <Satellite className="h-4.5 w-4.5" />
          </div>
          <div>
            <h3 className="text-sm font-bold text-slate-900">Live data feed – IBKR (module 0 · Eyes)</h3>
            <p className="text-xs text-slate-500">
              1-minute bars · session VWAP · RSI(14) · prior-day & session levels · refreshes every 60s during
              market hours
            </p>
          </div>
        </div>
        <div className="flex items-center gap-2 text-[11px] font-semibold">
          <span className={cn('rounded-full px-2.5 py-1', status?.marketOpen ? 'bg-emerald-50 text-emerald-700' : 'bg-slate-100 text-slate-500')}>
            {status?.marketOpen ? 'MARKET OPEN' : 'MARKET CLOSED'}
          </span>
        </div>
      </div>
    </section>
  );
}
```

```

    </span>
    <span className={cn('rounded-full px-2.5 py-1', status?.loopRunning ? 'bg-sky-50 text-sky-700' : 'bg-slate-100 text-slate-500')}>
      {status?.loopRunning ? `LOOP ${status.refreshIntervalSeconds}s` : 'LOOP OFF'}
    </span>
  </div>
</div>

<p className={cn('mb-4 text-[11px] font-medium', gatewayTone)}>
  Gateway: {status?.gatewayDetail ?? 'checking...'}
</p>

{/* watchlist input */}
<div className="mb-4 flex gap-2">
  <input
    value={symbol}
    onChange={(e) => setSymbol(e.target.value.toUpperCase())}
    onKeyDown={(e) => e.key === 'Enter' && submit()}
    placeholder="Add symbol to watchlist (e.g. SPY)"
    className="h-9 w-full max-w-xs rounded-lg border border-slate-200 bg-white/70 px-3 text-sm font-semibold text-slate-800 outline-none placeholder:font-normal placeholder:text-slate-400 focus:border-sky-400"
  />
  <button
    onClick={submit}
    disabled={trackMut.isPending || !symbol.trim()}
    className="inline-flex h-9 items-center gap-1.5 rounded-lg bg-gradient-to-r from-royal-500 to-teal-500 px-3.5 text-xs font-bold text-white transition hover:opacity-90 disabled:opacity-50"
  >
    <Plus className="h-3.5 w-3.5" /> Track
  </button>
</div>

{/* tracked symbols */}
{(!symbols || symbols.length === 0) && (
  <p className="rounded-xl border border-dashed border-slate-200 bg-white/50 p-4 text-center text-xs text-slate-500">
    No symbols tracked yet. Add a symbol above – the feed pulls 1-minute bars and computes indicators immediately, then keeps them fresh every 60 seconds while the market is open.
  </p>
)}
<div className="grid gap-3 md:grid-cols-2 xl:grid-cols-3">
  {(symbols ?? []).map((s) => (
    <div key={s.symbol} className="rounded-xl border border-slate-200/70 bg-white/60 p-4">
      <div className="mb-2 flex items-center justify-between">
        <span className="text-sm font-extrabold tracking-wide text-slate-900">{s.symbol}</span>
        <div className="flex items-center gap-1">
          <button
            onClick={() => refreshMut.mutate({ symbol: s.symbol })}
            className="rounded-md p-1.5 text-slate-400 transition hover:bg-slate-100 hover:text-slate-700"
            title="Refresh now"
          >
            <RefreshCw className="h-3.5 w-3.5" />
          </button>
          <button
            onClick={() => untrackMut.mutate({ symbol: s.symbol })}
            className="rounded-md p-1.5 text-slate-400 transition hover:bg-rose-50 hover:text-rose-600"
            title="Stop tracking"
          >
            <Trash2 className="h-3.5 w-3.5" />
          </button>
        </div>
      </div>
      <div>
        {s.error ? (
          <p className="text-[11px] font-medium leading-relaxed text-rose-600">{s.error}</p>
        ) : (
          <div className="grid grid-cols-3 gap-x-3 gap-y-1.5 text-[11px]">
            <Metric label="Last" value={fmt(s.last)} strong />
            <Metric label="VWAP" value={fmt(s.vwap)} />
            <Metric label="RSI-14" value={s.rsi14 !== null ? s.rsi14.toFixed(1) : '-'} />
            <Metric label="Bars" value={String(s.barCount)} />
            <Metric label="Delay" value={s.delaySeconds > 0 ? `${s.delaySeconds}s` : 'RT'} />
            <Metric

```

```

        label="Latency"
        value={s.latencyMs !== null ? `${s.latencyMs}ms` : '-'}
      />
    </div>
  )}
  {s.lastRefresh && (
    <p className="mt-2 flex items-center gap-1 text-[10px] text-slate-400">
      <Activity className="h-3 w-3" /> refreshed {new Date(s.lastRefresh).toLocaleTimeString()}
    </p>
  )}
</div>
)}}
</div>
</section>
);
}

function Metric({ label, value, strong }: { label: string; value: string; strong?: boolean }) {
  return (
    <div>
      <p className="text-[9px] font-bold uppercase tracking-wider text-slate-400">{label}</p>
      <p className={cn('font-mono font-semibold', strong ? 'text-slate-900' : 'text-slate-600')}>{value}</p>
    </div>
  );
}

function fmt(n: number | null): string {
  return n === null ? '-' : n.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDigits: 2 });
}

```

Appendix O — src/components/EnginePanel.tsx

Strategy Engine panel (React) — backtest, simulate, scan, monitor cards. — NEW FILE

```
import { useState } from 'react';
import { BrainCircuit, FlaskConical, Play, Radar, Square, Zap } from 'lucide-react';
import { trpc } from '@providers/trpc';
import { cn } from '@lib/utls';

/**
 * ENGINE PANEL — modules 1–6 of the autonomous strategy engine.
 * Backtester, what-if simulator, scanner, and live monitors that wire
 * CONFIRMED setups into the ticket confirmation gate (Phase 1: no
 * auto-execution — every proposal waits for CONFIRM ORDER [TICKET_ID]).
 */
export default function EnginePanel() {
  const utls = trpc.useUtls();
  const [symbol, setSymbol] = useState('');
  const [strategyId, setStrategyId] = useState('');

  const { data: configs } = trpc.engine.configs.useQuery();
  const { data: monitors } = trpc.engine.monitors.useQuery(undefined, { refetchInterval: 10000 });

  const backtestMut = trpc.engine.backtest.useMutation();
  const simulateMut = trpc.engine.simulate.useMutation();
  const scanMut = trpc.engine.scan.useMutation();
  const startMut = trpc.engine.monitorStart.useMutation({ onSettled: () => void
utls.engine.monitors.invalidate() });
  const stopMut = trpc.engine.monitorStop.useMutation({ onSettled: () => void utls.engine.monitors.invalidate()
});
  const evalMut = trpc.engine.evaluate.useMutation({
    onSettled: () => {
      void utls.engine.monitors.invalidate();
      void utls.execution.tickets.invalidate();
    },
  });

  const cfg = strategyId || configs?.[0]?.strategyId || '';
  const sym = symbol.trim().toUpperCase();
  const ready = sym.length > 0 && cfg.length > 0;

  const bt = backtestMut.data;
  const sim = simulateMut.data;
  const scan = scanMut.data;

  return (
    <section className="glass rounded-2xl p-6">
      <div className="mb-5 flex items-start gap-3">
        <div className="grid h-9 w-9 shrink-0 place-items-center rounded-xl bg-gradient-to-br from-royal-500 to-
teal-500 text-white">
          <BrainCircuit className="h-4.5 w-4.5" />
        </div>
        <div>
          <h3 className="text-sm font-bold text-slate-900">Strategy engine — modules 1–6 (Brain · Hands)</h3>
          <p className="text-xs text-slate-500">
            Trigger library · state machine · backtester · simulator · scanner · confirmation-gated execution.
            Configs are governance-clamped.
          </p>
        </div>
      </div>

      { /* controls */ }
      <div className="mb-5 flex flex-wrap items-center gap-2">
        <input
          value={symbol}
          onChange={(e) => setSymbol(e.target.value.toUpperCase())}
          placeholder="Symbol"
          className="h-9 w-28 rounded-lg border border-slate-200 bg-white/70 px-3 text-sm font-semibold outline-
none placeholder:font-normal focus:border-sky-400"
        />
        <select
```



```

    value={cfg}
    onChange={(e) => setStrategyId(e.target.value)}
    className="h-9 rounded-lg border border-slate-200 bg-white/70 px-3 text-xs font-semibold text-slate-700
outline-none focus:border-sky-400"
  >
    {(configs ?? []).map((c) => (
      <option key={c.strategyId} value={c.strategyId}>{c.label} (risk {c.riskPct}%)</option>
    ))}
  </select>
  <ActionBtn onClick={() => backtestMut.mutate({ symbol: sym, strategyId: cfg })} disabled={!ready ||
backtestMut.isPending} icon={<FlaskConical className="h-3.5 w-3.5" />} label={backtestMut.isPending ?
'Backtesting...' : 'Backtest'} />
  <ActionBtn onClick={() => simulateMut.mutate({ symbol: sym, strategyId: cfg })} disabled={!ready ||
simulateMut.isPending} icon={<Zap className="h-3.5 w-3.5" />} label={simulateMut.isPending ? 'Simulating...' :
'Simulate variants'} />
  <ActionBtn onClick={() => scanMut.mutate()} disabled={scanMut.isPending} icon={<Radar className="h-3.5 w-
3.5" />} label={scanMut.isPending ? 'Scanning...' : 'Scan watchlist'} />
  <ActionBtn onClick={() => startMut.mutate({ symbol: sym, strategyId: cfg })} disabled={!ready ||
startMut.isPending} icon={<Play className="h-3.5 w-3.5" />} label="Start monitor" primary />
</div>

{(backtestMut.error || simulateMut.error || startMut.error) && (
  <p className="mb-4 rounded-lg bg-rose-50 px-3 py-2 text-[11px] font-medium text-rose-700">
    {(backtestMut.error ?? simulateMut.error ?? startMut.error)?.message}
  </p>
)}

{/* backtest report */}
{bt && (
  <div className="mb-5 rounded-xl border border-slate-200/70 bg-white/60 p-4">
    <p className="mb-3 text-xs font-bold text-slate-800">
      Backtest – {bt.symbol} · {bt.days} sessions · governance {bt.governanceVersion}
      {bt.clamps.length > 0 && <span className="ml-2 font-medium text-amber-600">{bt.clamps.length} clamp
applied</span>}
    </p>
    <div className="mb-3 grid grid-cols-3 gap-3 sm:grid-cols-6">
      <Stat label="Trades" value={String(bt.totalTrades)} />
      <Stat label="Win rate" value={bt.winRate !== null ? `${bt.winRate}%` : '-'} />
      <Stat label="Expectancy" value={bt.expectancyR !== null ? `${bt.expectancyR}` : '-'} />
      <Stat label="Profit factor" value={bt.profitFactor !== null ? String(bt.profitFactor) : '-'} />
      <Stat label="Max DD" value={`${bt.maxDrawdownR}` } />
      <Stat label="No-trade days" value={String(bt.noTradeDays)} />
    </div>
    {bt.trades.length > 0 && (
      <table className="w-full text-left text-[11px]">
        <thead>
          <tr className="border-b border-slate-200 text-[9px] font-bold uppercase tracking-wider text-
slate-400">
            <th className="py-1.5 pr-2">Day</th><th className="pr-2">Var</th><th className="pr-
2">Entry</th><th className="pr-2">Avg exit</th><th className="pr-2">Qty</th><th className="pr-2">R</th><th
className="pr-2">P&L</th><th>Exit</th>
          </tr>
        </thead>
        <tbody>
          {bt.trades.slice(0, 8).map((t, i) => (
            <tr key={i} className="border-b border-slate-100 font-mono text-slate-600">
              <td className="py-1.5 pr-2">{t.day.slice(5)}</td>
              <td className="pr-2">{t.variantId}</td>
              <td className="pr-2">{t.entry.toFixed(2)}</td>
              <td className="pr-2">{t.avgExit.toFixed(2)}</td>
              <td className="pr-2">{t.qty}</td>
              <td className={cn('pr-2 font-bold', t.r > 0 ? 'text-emerald-600' : 'text-rose-600')}>
{t.r}</td>
              <td className={cn('pr-2 font-bold', t.pnl > 0 ? 'text-emerald-600' : 'text-rose-
600')}>${t.pnl}</td>
              <td>{t.exitReason}</td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </div>
)}

```

```

    {/* simulation ladder */}
    {sim && (
      <div className="mb-5 rounded-xl border border-slate-200/70 bg-white/60 p-4">
        <p className="mb-2 text-xs font-bold text-slate-800">What-if simulator – variant ladder</p>
        <div className="mb-3 grid gap-2 sm:grid-cols-3">
          {sim.variants.map((v) => (
            <div key={v.variantId} className={cn('rounded-lg border p-3 text-[11px]', v.kept ? 'border-emerald-200 bg-emerald-50/50' : 'border-slate-200 bg-slate-50 opacity-60')}>
              <p className="font-bold text-slate-800">Variant {v.variantId} · {v.triggerType}</p>
              <p className="font-mono text-slate-600">{v.trades} trades · exp {v.expectancyR ?? '-' }R · WR {v.winRate ?? '-'}%</p>
              <p className={cn('mt-1 font-semibold', v.kept ? 'text-emerald-700' : 'text-slate-500')}>{v.kept ? 'KEPT' : 'DROPPED – ${v.dropReason}`}</p>
            </div>
          ))}
        </div>
        <p className="text-[11px] font-medium text-slate-600">{sim.recommendation}</p>
      </div>
    )}

    {/* scanner candidates */}
    {scan && (
      <div className="mb-5 rounded-xl border border-slate-200/70 bg-white/60 p-4">
        <p className="mb-2 text-xs font-bold text-slate-800">Scanner – {scan.length} candidate(s) from tracked feeds</p>
        {scan.length === 0 && <p className="text-[11px] text-slate-500">Nothing met the gap/RVOL thresholds. Track more symbols or wait for market hours.</p>}
        <div className="grid gap-2 sm:grid-cols-2">
          {scan.map((c) => (
            <div key={c.symbol} className="flex items-center justify-between rounded-lg border border-slate-200 bg-white/70 p-3 text-[11px]">
              <div>
                <p className="text-xs font-extrabold text-slate-900">{c.symbol} <span className="font-mono font-medium text-slate-500">${c.last?.toFixed(2)}</span></p>
                <p className="text-slate-500">gap {c.gapPct ?? '-'}% · RVOL {c.relativeVolume ?? '-' } · vs VWAP {c.vsVwapPct ?? '-'}%</p>
                <p className="font-semibold text-sky-700">{c.suggestedLabel}</p>
              </div>
              <button
                onClick={() => startMut.mutate({ symbol: c.symbol, strategyId: c.suggestedConfig })}
                className="rounded-lg bg-gradient-to-r from-royal-500 to-teal-500 px-2.5 py-1.5 text-[10px] font-bold text-white hover:opacity-90"
              >
                Monitor
              </button>
            </div>
          ))}
        </div>
      </div>
    )}

    {/* active monitors */}
    <div className="rounded-xl border border-slate-200/70 bg-white/60 p-4">
      <p className="mb-2 text-xs font-bold text-slate-800">Live monitors – Phase 1 (confirmation-gated, no auto-execution)</p>
      {(!monitors || monitors.length === 0) && (
        <p className="text-[11px] text-slate-500">No monitors running. Start one above – CONFIRMED setups arrive as tickets requiring CONFIRM ORDER [TICKET_ID].</p>
      )}
      <div className="grid gap-2 sm:grid-cols-2">
        {(monitors ?? []).map((m) => (
          <div key={m.symbol} className="rounded-lg border border-slate-200 bg-white/70 p-3">
            <div className="flex items-center justify-between">
              <p className="text-xs font-extrabold text-slate-900">{m.symbol}</p>
              <span className={cn('rounded-full px-2 py-0.5 text-[9px] font-bold',
                m.state === 'CONFIRMED' || m.state === 'IN_POSITION' ? 'bg-emerald-100 text-emerald-700' :
                m.state === 'WATCHING' ? 'bg-sky-100 text-sky-700' :
                'bg-slate-100 text-slate-500')}>
                {m.state}{m.state === 'WATCHING' ? ` · T${m.timerCount}` : ''}
              </span>
            </div>
            <p className="mt-0.5 text-[10px] text-slate-500">{m.strategyId}</p>
          </div>
        ))}
      </div>
    </div>

```

```

    {evalMut.data && evalMut.data.symbol === m.symbol && (
      <p className="mt-1 text-[10px] font-medium text-slate-600">{evalMut.data.detail}</p>
    )}
    <div className="mt-2 flex gap-1.5">
      <button onClick={() => evalMut.mutate({ symbol: m.symbol })} disabled={evalMut.isPending}
        className="rounded-md bg-slate-800 px-2 py-1 text-[10px] font-bold text-white hover:bg-slate-
700 disabled:opacity-50">
        Evaluate now
      </button>
      <button onClick={() => stopMut.mutate({ symbol: m.symbol })}
        className="inline-flex items-center gap-1 rounded-md bg-rose-50 px-2 py-1 text-[10px] font-bold
text-rose-600 hover:bg-rose-100">
        <Square className="h-2.5 w-2.5" /> Stop
      </button>
    </div>
  </div>
)}
</div>
</div>
</section>
);
}

function ActionBtn({ onClick, disabled, icon, label, primary }: { onClick: () => void; disabled?: boolean; icon:
React.ReactNode; label: string; primary?: boolean }) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={cn(
        'inline-flex h-9 items-center gap-1.5 rounded-lg px-3 text-xs font-bold transition disabled:opacity-50',
        primary ? 'bg-gradient-to-r from-royal-500 to-teal-500 text-white hover:opacity-90' : 'border border-
slate-200 bg-white/70 text-slate-700 hover:border-sky-300',
      )}
    >
      {icon} {label}
    </button>
  );
}

function Stat({ label, value }: { label: string; value: string }) {
  return (
    <div className="rounded-lg bg-slate-50 p-2.5">
      <p className="text-[9px] font-bold uppercase tracking-wider text-slate-400">{label}</p>
      <p className="font-mono text-sm font-bold text-slate-900">{value}</p>
    </div>
  );
}

```

Appendix P — edits to existing files

api/router.ts: add the two imports and the two router keys; leave every existing router untouched.

```
import { marketDataRouter } from "../marketdata-router"; // NEW
import { engineRouter } from "../engine-router";          // NEW

export const appRouter = createRouter({
  // ...existing routers...
  execution: executionRouter,
  marketData: marketDataRouter, // NEW
  engine: engineRouter,          // NEW
});
```

src/pages/app/Autonomous.tsx – EDIT (mount the two panels)

Add the imports and render both panels directly after the RunConsole block in the page stack.

```
import DataFeedPanel from '@components/DataFeedPanel'; // NEW
import EnginePanel from '@components/EnginePanel';      // NEW

// inside the page stack, directly after the RunConsole block:
<div className="z-30 lg:sticky lg:top-[4.75rem]">
  <RunConsole mode={autonomyMode} />
</div>

<DataFeedPanel />    {/* NEW – module 0 live data feed */}
<EnginePanel />      {/* NEW – modules 1-6 strategy engine */}
```